VLSI DESIGN VERIFICATION

SystemVerilog Interview Preparation

Module 7: Inter-Process Communication and Synchronization

Detailed study notes for Design Verification roles

Concepts from first principles · complete SystemVerilog examples · line-by-line explanations
comparison tables · common mistakes · interview questions with answers · coding exercises

Contents

Module 7 Overview 4

Part 1: Why Inter-Process Communication Is Needed 5
- 1.1 Processes and Threads in SystemVerilog 5
- 1.2 Why Parallel Threads Must Communicate 7
- 1.3 The Simulation Time Slot: Why Order Within a Time Step Matters 10

Part 2: Events 13
- 2.1 What an Event Is: Declaration, Triggering and Waiting 13
- 2.2 wait(event.triggered) and the Trigger-versus-Wait Race 15
- 2.3 Nonblocking Event Trigger: ->> 19
- 2.4 Event Handles: Assignment, Merging, null and Comparison 21
- 2.5 Sharing Events Between Classes: Event-Based Communication 24
- 2.6 wait_order 27
- 2.7 Events: Summary, Mistakes and Questions 29

Part 3: Fork and Join 31
- 3.1 Sequential and Parallel Blocks 31
- 3.2 fork ... join 33
- 3.3 fork ... join_any 34
- 3.4 fork ... join_none 35
- 3.5 Differences Between join, join_any and join_none 36
- 3.6 Variables in Forked Threads and the Loop Trap 38
- 3.7 wait fork 40
- 3.8 disable fork and disable label 42
- 3.9 Nested fork-join 46
- 3.10 Parallel Process Execution and Synchronization Patterns 47

Part 4: Mailbox 50
- 4.1 What a Mailbox Is and Why It Is Required 50
- 4.2 Declaring and Creating a Mailbox: Bounded and Unbounded 52
- 4.3 Mailbox Methods 53
- 4.4 Bounded Mailbox in Action 56
- 4.5 Parameterized and Typeless Mailboxes 58
- 4.6 Mailbox of Transaction Objects 60
- 4.7 Communication Between Generator and Driver 62
- 4.8 Mailbox: Summary, Mistakes and Questions 66

Part 5: Semaphore 67
- 5.1 What a Semaphore Is and Why It Is Required 67
- 5.2 Declaration and Methods: new, get, put, try_get 69
- 5.3 Resource Sharing: Two Drivers on One Bus 71
- 5.4 Semaphore Pitfalls 74
- 5.5 Semaphore versus Mailbox, and Summary 75

Part 6: Timing and Synchronization 78
- 6.1 #delay versus @event versus wait 78
- 6.2 @event versus wait(event.triggered), and -> versus ->> 81
- 6.3 Blocking versus Nonblocking Synchronization 82
- 6.4 Race Conditions 83

6.5 Deadlocks 87
6.6 Synchronization Between Multiple Processes: a Complete Example 90
6.7 Event versus Mailbox versus Semaphore 92

Part 7: Module 7 Revision and Practice 95
7.1 Revision Summary 95
7.2 Interview Questions and Answers 97
7.3 Output-Prediction Questions 99
7.4 Find-the-Error and Debugging Questions 102
7.5 Coding Exercises with Solutions 106
7.6 Concepts You Must Understand Before Moving Forward 111
7.7 What Comes Next 112

Module 7 Overview

This module covers **inter-process communication (IPC) and synchronization**: how the parallel threads of a testbench signal each other, pass data to each other, and share resources without corrupting them. Interviewers treat this module as a test of whether you have really built a class-based testbench, because every such testbench is a set of threads (generator, driver, monitor, scoreboard) that must cooperate.

What this module covers

Part	Topic	Mechanism	Purpose
1	Why IPC is needed	Processes, scheduling, blocking	The problem that the other parts solve
2	Events	`event`, `->`, `->>`, `@`, `wait(ev.triggered)`, `wait_order`	"Something happened"
3	Fork and join	`fork` ... `join` / `join_any` / `join_none`, `disable fork`, `wait fork`	Creating and controlling parallel threads
4	Mailbox	`mailbox`, `put`, `get`, `peek`, `try_*`, `num`	Passing data between threads
5	Semaphore	`semaphore`, `get`, `put`, `try_get`	Sharing a resource safely
6	Timing and synchronization	`#`, `@`, `wait`, races, deadlocks	Using all of the above correctly
7	Revision and practice		Questions, traps, exercises

What you need from earlier modules

From	You should already know
M1	Data types, `logic` versus `bit`, enums, the ALU DUT and `alu_pkg`
M2	Queues (a mailbox is compared with a queue in Part 4)
M3	Tasks versus functions, `automatic` versus `static` lifetime, `ref` arguments. A task may block; a function may not. This rule decides which IPC methods can be called where
M4	Classes, handles, `new()`, `null`. Mailboxes and semaphores are built-in classes, and a mailbox of transactions stores **handles**
M6	The `alu_txn` transaction class

Naming used in this module. The examples use the names of the course testbench: transaction `alu_txn`, components `alu_generator` and `alu_driver`, mailboxes `gen2drv_mbx` and `mon2scb_mbx`, events `gen_done` and `drv_done`. So that every example compiles by itself, a cut-down `alu_txn` with the same field names is declared inside the example module, and field values are set by hand instead of being randomized. That keeps the printed output fixed.

> **Note:** The "Expected output" blocks were derived by hand from the IEEE 1800 rules, not captured from a simulator. The standard does **not** define the order in which threads that are ready at the same simulation time are executed. The examples are therefore written so that the output does not depend on that order: threads use different delays, or one line can only be printed after another because of a cause-and-effect chain. Where an example deliberately shows a race, the text says that the result depends on the simulator.

Part 1: Why Inter-Process Communication Is Needed

1.1 Processes and Threads in SystemVerilog

In plain English

Real hardware does everything at once: every flip-flop and every gate is active all the time. A testbench must imitate this. While the driver is wiggling the input pins, the monitor must be watching the output pins, the scoreboard must be checking results, and the clock must keep ticking. Each of these activities is written as a separate piece of code that runs "at the same time" as the others. Such a piece of code is called a **process** or a **thread**.

Technical definition

A **process** (also called a **thread** of execution) is an independently scheduled sequence of statements. A SystemVerilog simulation contains many processes. They are created by:

Construct	Creates
`initial` block	One process that starts at time 0 and runs once
`always`, `always_ff`, `always_comb`, `always_latch` block	One process that repeats forever
`final` block	One process that runs at the end of simulation (cannot block)
Continuous assignment (`assign`)	One process that re-evaluates when an input changes
Each statement inside `fork` ... `join` / `join_any` / `join_none`	One **dynamic** process per statement (Part 3)
Concurrent assertion	One process per attempt (M10)

Processes created by `initial`, `always` and `assign` exist for the whole simulation and are called **static processes**. Processes created by `fork` are created while the simulation runs and are called **dynamic processes**. A class-based testbench is built almost entirely from dynamic processes: the `run()` task of every component is started inside a `fork`.

How it works internally: there is only one CPU

The simulator is an ordinary program. It does not actually run the processes at the same time. It runs **one process at a time**, and switches to another process only when the running one **blocks**.

A process blocks when it executes a statement that must wait for something:

Blocking statement	Waits for
`#10;`	Simulation time to advance by 10 units
`@(posedge clk);`, `@(ev);`	A change on a signal, or an event trigger
`wait (expr);`	The expression to become true

Blocking statement	Waits for
`mbx.get(x);` on an empty mailbox	A message
`mbx.put(x);` on a full bounded mailbox	Free space
`sem.get();` with no key available	A key
`fork` ... `join`	All the child threads to finish
`wait fork;`	All the child threads to finish

```
   What the testbench describes            What the simulator actually does
   ----------------------------            --------------------------------

   generator  ========>                    time 0 :  run generator until it blocks
   driver     ========>                              run driver    until it blocks
   monitor    ========>                              run monitor   until it blocks
   scoreboard ========>                              run scoreboard until it blocks
                                                     nothing left to run at time 0
   (four threads "in parallel")            time 5 :  wake the threads waiting for time 5,
                                                     run each until it blocks ...
```

Three consequences follow, and all of IPC rests on them.

1. **Within one simulation time, threads run one after another, in an order the standard does not define.** If two threads are both ready at time 10, the simulator may run either first. Code whose result depends on that order has a **race condition**.
2. **A thread that never blocks never lets any other thread run.** A `forever` loop with no delay, event or blocking call inside it hangs the simulation at the current time.
3. **Simulation time advances only when every thread is blocked.** Then the simulator moves to the next time at which something is scheduled.

Complete example

tb_processes.sv

```sv
module tb_processes;

  initial begin : thread_a
    $display("[%0t] A: step 1", $time);
    #10;
    $display("[%0t] A: step 2", $time);
    #20;
    $display("[%0t] A: step 3", $time);
  end

  initial begin : thread_b
    #5;
    $display("[%0t] B: step 1", $time);
    #10;
    $display("[%0t] B: step 2", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] A: step 1
[5] B: step 1
[10] A: step 2
[15] B: step 2
[30] A: step 3
```

Line by line

Time	What happens
0	Both `initial` blocks become ready. `thread_a` prints and blocks at `#10`. `thread_b` blocks at `#5` without printing. Whichever the simulator runs first, the output is the same, because only one of them prints at time 0.
5	Every thread is blocked, so time advances to the earliest wake-up, 5. `thread_b` prints and blocks until 15.
10	`thread_a` wakes, prints, and blocks until 30.
15	`thread_b` wakes, prints, and ends.
30	`thread_a` wakes, prints, and ends. No process is left, so the simulation stops.

The two threads interleave. Neither knows anything about the other. That is fine here because they do not share anything. The rest of this module is about what happens when they do.

Interview questions

Q1. What is a process (thread) in SystemVerilog? An independently scheduled sequence of statements. `initial` and `always` blocks and continuous assignments create static processes; each statement of a `fork` block creates a dynamic process.

Q2. Do SystemVerilog threads really run in parallel? No. The simulator executes one thread at a time. A thread runs until it reaches a blocking statement (a delay, an event control, a `wait`, or a blocking method call), and then another ready thread runs. The parallelism is in simulation time, not in real time.

Q3. Two `initial` blocks both start at time 0. Which one runs first? The standard does not say. Code must not depend on the order.

Q4. What happens if a `forever` loop contains no blocking statement? The thread never gives up control. Simulation time cannot advance and the simulation hangs (a "zero-delay loop").

1.2 Why Parallel Threads Must Communicate

In plain English

Consider the two most basic testbench components.

- The **generator** creates transactions (what to send to the DUT).
- The **driver** takes a transaction and turns it into pin wiggles on the DUT inputs, which takes one or more clock cycles.

They are separate classes and run as separate threads, for a good reason: the generator knows nothing about pins and clocks, and the driver knows nothing about what makes a good test. But they cannot work independently:

1. The generator must **hand each transaction** to the driver. → data transfer
2. The generator must **not produce faster** than the driver can drive, or transactions are lost or pile up without limit. → flow control
3. The generator may need to know **when the driver has finished** an item, and the test must know **when everything is done** so that it can end. → notification

4. If two drivers use the **same bus**, only one may drive at a time. → mutual exclusion

SystemVerilog provides one construct for each need.

Need	Question it answers	Construct	Part
Notification	"Has X happened yet?"	`event`	2
Creating and joining threads	"Start these in parallel; continue when all / one / none have finished"	`fork` ... `join` / `join_any` / `join_none`	3
Data transfer with flow control	"Give this item to the other thread; wait if it is not ready"	`mailbox`	4
Mutual exclusion	"May I use the shared resource now?"	`semaphore`	5

Together these are called the **inter-process communication (IPC)** and **synchronization** constructs.

- **Communication** means passing information from one thread to another.
- **Synchronization** means making threads wait for each other so that things happen in the right order.

What goes wrong without IPC: a shared variable

The obvious first idea is a shared variable: the producer writes it, and the consumer reads it. The example below shows why that is not enough.

tb_no_ipc.sv

```
module tb_no_ipc;

  int shared_data;
  bit new_data;

  initial begin : producer
    for (int i = 1; i <= 4; i++) begin
      $display("[%0t] producer writes %0d", $time, i * 11);
      shared_data = i * 11;
      new_data    = 1;
      #5;                              // produces one item every 5 time units
    end
  end

  initial begin : consumer
    forever begin
      wait (new_data);                 // wait until something has been written
      new_data = 0;
      $display("[%0t] consumer reads  %0d", $time, shared_data);
      #12;                             // needs 12 time units to process one item
    end
  end

endmodule
```

EXPECTED OUTPUT

```
[0] producer writes 11
[0] consumer reads  11
[5] producer writes 22
[10] producer writes 33
[12] consumer reads  33
[15] producer writes 44
[24] consumer reads  44
```

Execution flow

Time	Producer	Consumer	`shared_data`
0	Writes 11, sets `new_data`	`wait (new_data)` passes, reads 11, starts processing until 12	11
5	Writes 22	Busy	22
10	Writes 33. **22 is overwritten before anybody read it**	Busy	33
12		Finishes, `new_data` is still 1, reads 33, busy until 24	33
15	Writes 44, then ends	Busy	44
24		Reads 44, busy until 36	44
36		`wait (new_data)` blocks forever. Nothing else is scheduled, so the simulation ends	

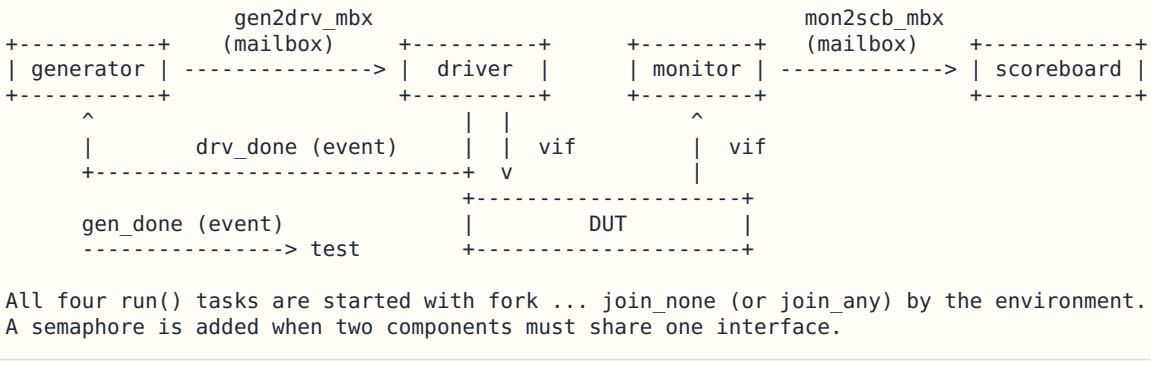
Four items were produced and only three were consumed. Item 22 was silently lost. The shared variable has three defects:

Defect	Why
No storage	One variable holds one value. A second write destroys the first.
No flow control	The producer has no way to know that the consumer is still busy.
Hand-written handshake	The `new_data` flag is a home-made protocol. With more than two threads it becomes very easy to get wrong.

A mailbox (Part 4) removes all three: it stores any number of items in order, it can make the producer wait when it is full, and the handshake is built in.

> **Key point:** IPC constructs exist because threads run independently and at different speeds. A plain shared variable gives no storage, no waiting and no protection. Events add waiting, mailboxes add storage and waiting, semaphores add protection.

Where IPC appears in a layered testbench



Every arrow in this picture is one of the constructs of this module. Module 9 builds exactly this structure for the ALU.

Interview questions

Q1. What is inter-process communication and why is it needed in a testbench? IPC is the set of mechanisms by which parallel threads exchange information and coordinate their execution. A testbench consists of components that run as separate threads (generator, driver, monitor, scoreboard). They must pass transactions to each other, wait for each other, and share resources such as an interface. Events, mailboxes and semaphores provide this; `fork`...`join` creates the threads.

Q2. Why not use a shared variable or a queue between the generator and the driver? A shared variable holds only one value, so a fast producer overwrites data, and it gives the consumer no way to wait for data except polling. A queue stores many items but has no blocking: the consumer must be written with its own `wait (q.size() > 0)` and the producer has no flow control. A mailbox has storage, blocking `get` and optional blocking `put` built in.

Q3. Name the SystemVerilog IPC constructs and the purpose of each. `event` for notification that something happened; `mailbox` for passing data between threads; `semaphore` for controlling access to a shared resource; and `fork`...`join`, `join_any`, `join_none` with `wait fork` and `disable fork` for creating and controlling the threads.

Q4. What is the difference between communication and synchronization? Communication transfers data between threads (a mailbox). Synchronization controls the relative timing of threads so that actions occur in the required order (events, semaphores, joining threads). A mailbox does both: `get` blocks until data arrives.

1.3 The Simulation Time Slot: Why Order Within a Time Step Matters

To understand races, `->` versus `->>`, and `@` versus `wait(triggered)`, you need a simplified picture of how the simulator processes one moment of simulation time. Everything that happens at one simulation time is called a **time slot**. A time slot is divided into ordered **regions**.

```
One time slot (all activity at a single simulation time)

     Preponed   : sample values for assertions and clocking blocks (nothing changes here)
+--> Active     : run ready processes: blocking assignments, -> ev, $display,
|                 evaluate the right-hand side of <=
|    Inactive   : processes that executed #0
|    NBA        : update the left-hand side of <= ; perform ->> ev triggers
|    Observed   : evaluate concurrent assertions
|    Reactive   : program-block code, assertion action blocks, clocking-block drives
+--- loop back while any of these regions still has something to do
     Postponed  : $strobe and $monitor print; nothing may change any more

Then simulation time advances to the next time at which something is scheduled.
```

The points to remember for this module:

Fact	Consequence
All ready threads run in the **Active** region, in an undefined order	Two threads that touch the same variable or event at the same time can race
A plain trigger `-> ev` takes effect immediately, in the Active region	A thread that starts waiting later in the same time slot has missed it
A nonblocking trigger `->> ev` takes effect in the **NBA** region	Every thread that ran in the Active region is already waiting when the trigger arrives

Fact	Consequence
`ev.triggered` stays true from the trigger until the **end of the time slot**	`wait (ev.triggered)` cannot miss a trigger made earlier in the same time slot
`#0` moves a thread to the **Inactive** region	The thread runs again after all other Active threads have run. It is a crude way of saying "let the others go first" and is discouraged
The regions repeat (loop back)	A thread woken by an NBA update runs in a new Active region of the same time slot

The full scheduling model, including the Reactive region and `program` blocks, belongs to Module 8 (clocking blocks) and Module 11. This simplified picture is enough for IPC.

The words used in this module

Term	Meaning
Thread / process	An independent sequence of statements
Parent / child	The thread that executes a `fork`, and the threads that the `fork` creates
Blocking statement	A statement that may suspend the thread until something happens
Nonblocking call	A call that never suspends the thread; it reports success or failure immediately (`try_get`, `try_put`)
Time step / time slot	All activity at one value of simulation time
Race condition	The result depends on the order in which threads run within one time step
Deadlock	Two or more threads each wait for something only another waiting thread can provide, so none can ever continue
Mutual exclusion	Only one thread at a time may use a resource
Critical section	The code that uses the shared resource, between acquiring and releasing it
Handshake	Two threads signalling each other so that each knows the other has completed a step

Interview questions

Q1. What is a time slot? All the simulation activity that occurs at one simulation time. It is processed in ordered regions (Active, Inactive, NBA, Observed, Reactive, Postponed), looping until no events remain, before time advances.

Q2. In which region is a blocking assignment executed, and in which region is the left-hand side of a nonblocking assignment updated? A blocking assignment is executed in the Active region. For a nonblocking assignment, the right-hand side is evaluated in the Active region and the left-hand side is updated in the NBA region.

Q3. What does `#0` do? It suspends the thread and reschedules it in the Inactive region of the same time slot, so it resumes after the other Active threads have run. It is sometimes used to "fix" a race, but it only hides the ordering problem; if another thread also uses `#0` the race returns. Proper synchronization (events, nonblocking triggers, mailboxes, clocking blocks) is preferred.

Q4. What is a race condition? A situation in which the outcome of the simulation depends on the order in which the simulator executes threads that are ready at the same simulation time. Because that order is not defined by

the standard, the same code may give different results on different simulators, with different options, or after an unrelated code change.

Part 2: Events

2.1 What an Event Is: Declaration, Triggering and Waiting

In plain English

An event is a doorbell. One thread presses it; every thread that is standing at the door listening at that moment hears it and continues. The bell carries no message: it only says "now". And it does not keep ringing: a thread that arrives after the bell was pressed hears nothing and waits for the next press.

Technical definition

An **event** (also called a **named event**) is a synchronization object declared with the keyword `event`. It has no value and no duration. Two operations are defined on it:

- **Triggering** it with `->` (or `->>`), which unblocks every process that is waiting for it at that instant.
- **Waiting** for it with the event control `@`, or with `wait (ev.triggered)`.

In SystemVerilog an event variable is a **handle** to the underlying synchronization object. It can be assigned, compared, passed as an argument and set to `null` (Section 2.4).

Why it is needed

Threads often need to know that something has happened, without any data being exchanged:

- reset has finished, so the driver may start;
- the driver has finished one transaction, so the generator may create the next;
- the generator has produced its last transaction, so the test may start its end-of-test checks;
- the monitor has seen a complete packet, so the coverage collector should sample.

Polling a flag variable in a loop would work but wastes simulation effort and is easy to get wrong. An event lets a thread sleep until exactly the moment it is needed.

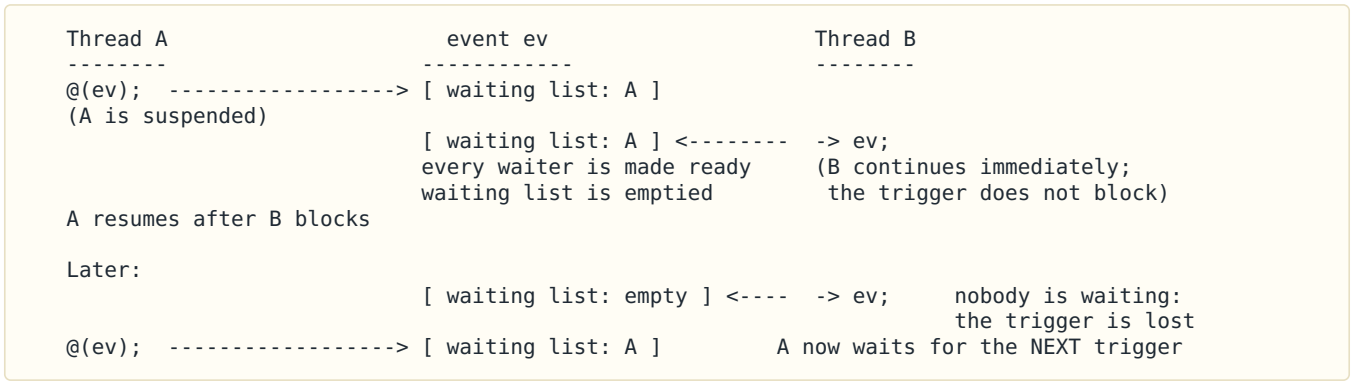
Syntax

```
event ev;                 // declaration: creates an event object, ready to use (no new())

-> ev;                    // trigger: wake every thread waiting on ev right now
->> ev;                   // nonblocking trigger: happens in the NBA region (Section 2.3)

@(ev);                    // wait for the next trigger of ev      (also written  @ev; )
@(ev1 or ev2);            // wait for whichever triggers first
wait (ev.triggered);      // wait unless ev has already triggered in this time step
```

How it works internally



Three rules follow from this picture.

1. A trigger is an instant. It affects only the threads that are **already waiting**.
2. `-> ev` does not block the triggering thread. The woken threads run after the triggering thread itself blocks or ends.
3. One trigger wakes **all** waiting threads. The order in which they then run is not defined.

Complete example

tb_event_basic.sv

```
module tb_event_basic;

  event ev;

  initial begin : waiter
    $display("[%0t] waiter : waiting for ev", $time);
    @(ev);
    $display("[%0t] waiter : ev received", $time);
    @(ev);
    $display("[%0t] waiter : ev received again", $time);
  end

  initial begin : trigger
    #10;
    $display("[%0t] trigger: -> ev", $time);
    -> ev;
    #10;
    $display("[%0t] trigger: -> ev", $time);
    -> ev;
  end

endmodule
```

EXPECTED OUTPUT

```
[0] waiter : waiting for ev
[10] trigger: -> ev
[10] waiter : ev received
[20] trigger: -> ev
[20] waiter : ev received again
```

Line by line

Line	Explanation
`event ev;`	Declares the event. Unlike a mailbox or semaphore, no `new()` is needed.

Line	Explanation
`@(ev);` (first)	The waiter is suspended and placed on the waiting list of `ev`. This happens at time 0, long before the first trigger.
`#10; ... -> ev;`	At time 10 the trigger thread prints and then triggers. The waiter becomes ready. The trigger thread is not suspended by `->`; it continues to its own `#10`.
`ev received`	The waiter runs once the trigger thread has blocked. The time is still 10.
`@(ev);` (second)	The waiter waits again. The trigger at time 10 is over; this `@` waits for the **next** one, at time 20.

The waiter's line at time 10 always comes after the trigger's line, because the trigger thread prints *before* it executes `->`, and the waiter cannot run before the trigger.

> **Common mistake:** Expecting an event to "remember" that it was triggered. It does not. `-> ev;` followed later by `@(ev);` waits for a second trigger. If you need a memory, use `wait (ev.triggered)` within the same time step, or a flag variable, or a mailbox or semaphore, which do store their state.

> **Verification application:** Events are used for phase-like notifications in a testbench: `reset_done`, `gen_done`, `drv_done`, `end_of_test`. In UVM the same idea appears as `uvm_event`, which adds a persistent "on" state and the ability to carry data.

Interview questions

Q1. What is an event in SystemVerilog? A synchronization object, declared with `event`, that one process triggers and other processes wait for. It carries no data and has no duration.

Q2. How do you trigger an event and how do you wait for it? Trigger with `-> ev;` (or `->> ev;` for a nonblocking trigger). Wait with `@(ev);` or `wait (ev.triggered);`.

Q3. Does `-> ev` block the process that executes it? No. The triggering process continues with its next statement. The waiting processes become ready and run when the triggering process blocks.

Q4. If three processes are waiting on the same event and it is triggered once, how many wake up? All three. The order in which they execute is not defined.

Q5. Does an event need to be constructed with `new()`? No. Declaring an event variable creates the event object. (Mailboxes and semaphores are classes and do need `new()`.)

2.2 wait(event.triggered) and the Trigger-versus-Wait Race

The problem

`@(ev)` only sees triggers that occur **after** it has started waiting. If the trigger and the `@` are executed at the same simulation time by two different threads, the simulator may run either thread first:

```
event ev;

initial begin : trigger
  #10 -> ev;
end

initial begin : waiter
  #10 @(ev);
  $display("[%0t] waiter: ev received", $time);     // printed, or not?
end
```

```
   Possibility 1: the simulator runs "waiter" first at time 10

      time 10, Active region :  waiter  executes @(ev)   -> starts waiting
                                trigger executes -> ev   -> waiter is woken
                                waiter  prints                              OK

   Possibility 2: the simulator runs "trigger" first at time 10

      time 10, Active region :  trigger executes -> ev   -> nobody is waiting: lost
                                waiter  executes @(ev)   -> starts waiting ... forever
                                                                            HANGS
```

Both possibilities are legal. Which one you get depends on the simulator, its version, its options, and even on the order of the `initial` blocks in the file. This is a **race condition**, and it is one of the most common causes of "the test hangs on one simulator and passes on another".

The solution: the triggered property

Every event has a built-in property `triggered`.

- `ev.triggered` is **true from the moment the event is triggered until the end of the current time step**, and false otherwise.
- `wait (ev.triggered);` is therefore a **level-sensitive** wait: if the event has already been triggered in this time step, it does not block; otherwise it blocks until the trigger.

```
   time 10                                              time 11
   |<------------------ one time step ----------------->|
          -> ev
           |
   ev.triggered :  0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 | 0 0 0 0 ...
                        ^                     ^
                        |                     |
                 trigger happens      a wait (ev.triggered) executed anywhere
                                      in here passes immediately

   @(ev) passes only if it was already waiting at the instant marked "trigger happens".
```

With `wait (ev.triggered)` the race above disappears: whichever thread runs first, the waiter continues at time 10.

Complete example

To make the output independent of the simulator, the example puts the trigger and the two kinds of wait in a known order inside one parent thread.

tb_event_triggered.sv

```
module tb_event_triggered;

  event ev;

  initial begin
    -> ev;                                      // the trigger happens FIRST, at time 0

    fork                                        // both waits start AFTER the trigger
      begin
        @(ev);
        $display("[%0t] thread 1: @(ev) passed", $time);
      end
      begin
        wait (ev.triggered);
        $display("[%0t] thread 2: wait (ev.triggered) passed", $time);
      end
    join_none

    #20;
    $display("[%0t] parent  : -> ev again", $time);
    -> ev;
  end

endmodule
```

EXPECTED OUTPUT

```
[0] thread 2: wait (ev.triggered) passed
[20] parent  : -> ev again
[20] thread 1: @(ev) passed
```

Line by line

Line	Explanation
`-> ev;`	Triggered at time 0. Nobody is waiting yet. `ev.triggered` is now true for the rest of time 0.
`fork ... join_none`	Creates two child threads. With `join_none` they start running only when the parent blocks, which is at `#20`. The time is still 0.
Thread 1: `@(ev);`	The trigger is already in the past. Thread 1 waits for the next one.
Thread 2: `wait (ev.triggered);`	`ev.triggered` is still true at time 0, so thread 2 does not block and prints immediately.
`#20; ... -> ev;`	At time 20 the event is triggered again. Thread 1 has been waiting since time 0, so it wakes now.

@(ev) versus wait(ev.triggered)

Property	`@(ev)`	`wait (ev.triggered)`
Sensitivity	Edge: the instant of the trigger	Level: true for the rest of the time step
Trigger happened earlier in the same time step	**Missed**; waits for the next trigger	**Seen**; does not block
Trigger happened in an earlier time step	Missed	Missed (the property was cleared when time advanced)
Race with a trigger in the same time step	Yes	No

Property	`@(ev)`	`wait (ev.triggered)`
Safe inside a loop that consumes no time	Yes, each `@` needs a new trigger	**No**: passes again and again in the same time step (infinite loop)
Typical use	Repeated notifications in a loop	One-shot notifications such as "reset done", "generator done"

Interview trap: "Is `wait (ev.triggered)` always better than `@(ev)`?" No. In a loop that does not advance time it is dangerous:

```
forever begin wait (ev.triggered); count++; end
```

After the first trigger, `ev.triggered` stays true for the whole time step, so the loop spins forever at that time and the simulation hangs. Inside loops use `@(ev)`, or make sure time advances in every iteration.

Common mistake: Writing `@(ev.triggered)`. That waits for the *value* of the property to change, which is not what is wanted and reintroduces the race. The correct form is `wait (ev.triggered)`.

Key point: `ev.triggered` does not make an event permanent. It only covers the remainder of the time step in which the trigger occurred. A waiter that arrives one time unit later has still missed it.

Interview questions

Q1. What is the difference between `@(ev)` and `wait (ev.triggered)`? `@(ev)` is edge-sensitive: it unblocks only if it was already waiting when the event was triggered. `wait (ev.triggered)` is level-sensitive within the time step: it unblocks immediately if the event has been triggered at any earlier point in the current time step, and otherwise waits for the trigger. The second form removes the race between a trigger and a wait that occur at the same simulation time.

Q2. An event is triggered at time 10 and a process executes `@(ev)` at time 10. Does the process continue? It depends on which executes first, and that order is not defined. If the `@` executes first, the process continues at time 10. If the trigger executes first, the process waits for the next trigger. Using `wait (ev.triggered)` makes the result deterministic.

Q3. An event is triggered at time 10 and a process executes `wait (ev.triggered)` at time 15. Does the process continue? No. The `triggered` property is cleared when simulation time advances beyond the time step of the trigger. The process waits for the next trigger.

Q4. Why can `wait (ev.triggered)` cause a simulation to hang? If it is used in a loop that contains no other time-consuming statement, the condition remains true for the whole time step after the first trigger, so the loop iterates endlessly without letting time advance.

2.3 Nonblocking Event Trigger: ->>

In plain English

`-> ev` rings the bell right now. `->> ev` says "ring the bell a moment later, after everybody who is running at this time has had the chance to reach the door". The thread that executes `->>` does not wait; it only schedules the ringing.

Technical definition

The **nonblocking event trigger** `->> ev;` schedules the event to be triggered in the **NBA (nonblocking assignment) region** of the current time step, in the same way that `a <= b;` schedules an update of `a`. The statement itself completes immediately. An optional delay or event control may be given: `->> #5 ev;` triggers the event five time units later, again without blocking the thread that executed it.

Why it is needed

It is the second way of removing the trigger-versus-wait race, applied on the triggering side. All threads that are ready at a given time run in the Active region. A thread that executes `@(ev)` in the Active region is waiting by the time the NBA region is reached, so a trigger scheduled there cannot be missed.

```
time 10
Active region :   trigger thread :  ->> ev        (only scheduled, nothing happens yet)
                  waiter thread  :  @(ev)         (starts waiting)
                  -- the order of these two no longer matters --
NBA region    :   ev is triggered  ->  waiter is woken
Active region :   waiter runs (still time 10)
```

Complete example 1: the race removed

tb_event_nonblocking.sv

```systemverilog
module tb_event_nonblocking;

  event ev;

  initial begin : trigger
    #10;
    ->> ev;                 // takes effect in the NBA region of time 10
  end

  initial begin : waiter
    #10;
    @(ev);                  // starts waiting in the Active region of time 10
    $display("[%0t] waiter: ev received", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[10] waiter: ev received
```

This is the racy example of Section 2.2 with one character added. Whichever `initial` block the simulator runs first at time 10, the `@(ev)` has been executed before the NBA region, so the waiter always continues.

Complete example 2: -> versus ->> in a single thread

tb_event_nb_single.sv

```
module tb_event_nb_single;

  event ev;

  initial begin
    ->> ev;
    $display("[%0t] 1. after ->> ev   : statement did not block", $time);
    @(ev);
    $display("[%0t] 2. @(ev) passed   : the trigger arrived in the NBA region", $time);

    ->> #15 ev;
    $display("[%0t] 3. after ->> #15 ev : statement did not block", $time);
    @(ev);
    $display("[%0t] 4. @(ev) passed", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] 1. after ->> ev   : statement did not block
[0] 2. @(ev) passed   : the trigger arrived in the NBA region
[0] 3. after ->> #15 ev : statement did not block
[15] 4. @(ev) passed
```

Line by line

Line	Explanation
`->> ev;`	Schedules a trigger for the NBA region of time 0. The thread continues at once.
`@(ev);` (first)	The thread starts waiting. When the NBA region is reached the event fires and the thread resumes, still at time 0. If the first statement had been `-> ev;`, this `@` would have waited forever: a thread cannot catch its own blocking trigger.
`->> #15 ev;`	Schedules a trigger for time 15. The thread is **not** delayed: line 3 is printed at time 0.
`@(ev);` (second)	Waits until the scheduled trigger at time 15.

-> versus ->>

Property	`-> ev`	`->> ev`
Name	Blocking trigger (immediate trigger)	Nonblocking trigger
When the event fires	Immediately, in the Active region	In the NBA region of the same time step (or after the given delay)
Does the statement suspend the thread	No	No
A thread that executes `@(ev)` later in the same Active region	Misses it	Catches it
Optional delay or event control	Not allowed	Allowed: `->> #5 ev;`, `->> @(posedge clk) ev;`
Analogy	Blocking assignment `=`	Nonblocking assignment `<=`

The names are confusing: neither form blocks the thread that executes it. "Blocking" and "nonblocking" here follow the naming of the two assignment operators and describe *when the effect takes place*.

Three ways to avoid the trigger-versus-wait race

Method	Change made on	How it works
`wait (ev.triggered)`	The waiting side	The trigger is remembered until the end of the time step
`->> ev`	The triggering side	The trigger is postponed to the NBA region
Design the protocol so that the waiter is always waiting first	Both	For example the waiter starts at time 0 and the trigger can only occur after a clock edge

> **Common mistake:** Using both `->>` and a waiter that is itself woken in the NBA region. If the waiter only reaches its `@(ev)` *after* an NBA update (for example it was waiting on a signal assigned with `<=`), it executes `@` after the trigger and still misses it. `->>` protects only waiters that run in the Active region.

Interview questions

Q1. What is the difference between `->` and `->>`? `->` triggers the event immediately. `->>` schedules the trigger for the nonblocking-assignment region of the current time step (or for a later time if a delay is given). With `->>`, a process that starts waiting with `@` in the same time step, even after the `->>` statement was executed, still receives the trigger.

Q2. Does `->>` block the calling process? No. Neither `->` nor `->>` suspends the process that executes it.

Q3. What does `->> #10 ev;` do? It schedules `ev` to be triggered ten time units from now and continues immediately. `#10 -> ev;` would instead suspend the process for ten time units and then trigger.

Q4. A process executes `-> ev; @(ev);`. What happens? What if the first statement is `->> ev;`? With `->`, the process waits forever (unless another process triggers `ev`), because the trigger is already over when `@` starts waiting. With `->>`, the trigger occurs in the NBA region after the process has started waiting, so the process continues in the same time step.

2.4 Event Handles: Assignment, Merging, null and Comparison

In plain English

In Verilog a named event was a fixed object. In SystemVerilog the name you declare is a **handle**: a reference to an event object. Handles can be copied. After `b = a;` the two names refer to the same bell, so pressing either name rings it and listening on either name hears it. This is what makes it possible to share one event between objects of different classes.

Technical definition

- `event a;` creates a new event object and makes `a` refer to it.

- `event b = a;` or `b = a;` makes `b` refer to the object of `a`. The two events are said to be **merged** (aliased). Triggering either one unblocks processes waiting on either one.
- `a = null;` breaks the association. `a` then refers to no object.
- Event handles can be compared with `==`, `!=`, `===`, `!==`, and with `null`. Two handles are equal if they refer to the same object.
- An event can be an argument of a task or function, a class property, an element of an array, and the value returned by a function.

Memory picture

```
event a, b, c;                a ---> [ event object 1 ]
                              b ---> [ event object 2 ]
                              c ---> [ event object 3 ]

b = a;                        a ---> [ event object 1 ] <--- b
                                     [ event object 2 ]   (unreachable through b;
                                                           threads already waiting on it
                                                           stay there forever)
c = null;                     c ---> (nothing)
```

Rules

Operation	Behaviour
`b = a;` then `-> a;`	Wakes processes waiting on `a` **and** processes that started waiting on `b` after the assignment
A process is already blocked in `@(b)` when `b = a;` is executed	That process stays attached to the **old** object of `b`. It never unblocks, because nothing can trigger the old object through `b` any more
`-> c;` when `c` is `null`	No effect
`@(c);` or `wait (c.triggered);` when `c` is `null`	Undefined by the standard: it may block forever or not block at all. Never do this
`a == b`	1 if both refer to the same event object
`if (c == null)`	Test for an unassigned event handle

Complete example

tb_event_handles.sv

```
module tb_event_handles;

  event a, b, c;

  initial begin
    $display("1. a == b : %b   (two separate event objects)", a == b);

    b = a;                                  // merge: b now refers to the object of a
    $display("2. a == b : %b   (after b = a)", a == b);

    fork
      begin
        @(b);
        $display("[%0t] 3. the waiter on b was woken by -> a", $time);
      end
    join_none

    #10 -> a;

    #10;
    c = null;
    $display("[%0t] 4. c == null : %b", $time, c == null);
    -> c;                                   // triggering a null event does nothing
    $display("[%0t] 5. -> c on a null event had no effect", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
1. a == b : 0   (two separate event objects)
2. a == b : 1   (after b = a)
[10] 3. the waiter on b was woken by -> a
[20] 4. c == null : 1
[20] 5. -> c on a null event had no effect
```

Line by line

Line	Explanation
1	Each declared event has its own object, so the handles differ.
`b = a;`	Handle copy. No new object is created.
2	Both handles now refer to the same object.
`@(b);`	The child starts waiting when the parent blocks at `#10`. It waits on the object that `b` refers to **now**, which is the object of `a`.
`#10 -> a;`	Triggering through `a` wakes the waiter that used the name `b`.
`c = null;` and `-> c;`	A null event can be triggered without error, with no effect.

The merging hazard

```
event a, b;

initial begin
  fork
    begin
      @(b);                          // (1) starts waiting on the ORIGINAL object of b
      $display("never printed");
    end
  join_none
  #1;
  b = a;                             // (2) b now refers to the object of a
  -> b;                              // (3) triggers the object of a, not the one in (1)
end
```

The child thread is attached to the object that `b` referred to at step (1). After step (2) no handle refers to that object, so it can never be triggered and the thread is stuck. **Assign shared event handles before starting the threads that wait on them.** In a testbench this means: connect the events in the environment's constructor or build step, and only then `fork` the `run()` tasks.

> **Interview trap:** "What happens to a process waiting on an event when that event variable is assigned another event?" It never unblocks. The assignment only affects event controls and waits that are executed *after* it.

Interview questions

Q1. Is an event variable a handle? Yes. In SystemVerilog an event variable refers to an underlying synchronization object. It can be assigned to another event variable, compared, passed as an argument and assigned `null`.

Q2. What does `event b = a;` do? It makes `b` an alias of `a`: both refer to the same event object. Triggering either wakes the processes waiting on either.

Q3. What happens if you trigger a null event? Wait on a null event? Triggering has no effect. Waiting on a null event is undefined by the standard and must be avoided.

Q4. How can you tell whether two event variables refer to the same event? Compare them: `if (a == b)`.

Q5. What is the difference between a Verilog named event and a SystemVerilog event? A Verilog named event is a static object that can only be triggered with `->` and waited for with `@`. A SystemVerilog event is a handle: it adds assignment and merging, `null`, comparison, passing to tasks and classes, the `triggered` property, the nonblocking trigger `->>` and `wait_order`.

2.5 Sharing Events Between Classes: Event-Based Communication

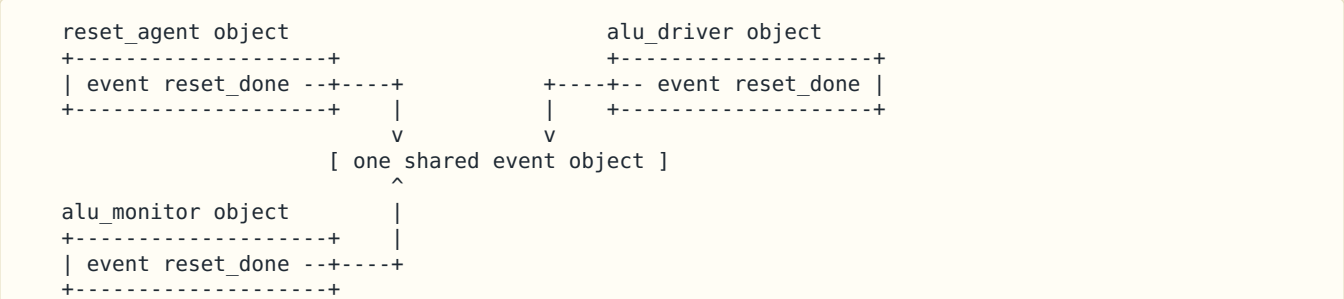
The problem

A driver object must not start driving until reset has finished. The reset is generated by a different object. The two objects are instances of different classes, created separately; neither can see the local variables of the other. How does the driver wait for something that happens inside the reset agent?

The solution

Because an event variable is a handle, two objects can each hold a handle to **the same** event object. There are three common ways to arrange this.

Method	Code	Notes
Assign the handle after construction	`drv.reset_done = rst.reset_done;`	Simple. Must be done before `run()` starts
Pass the event to the constructor	`drv = new(reset_done);` with `function new(event reset_done);`	Cannot be forgotten: the object cannot be built without it
Reach through an object handle	`@(env.gen.gen_done);`	Convenient in the test or environment, which already hold handles to the components



Complete example

tb_event_classes.sv

```
module tb_event_classes;

  class reset_agent;
    event reset_done;

    task run();
      $display("[%0t] RST: asserting reset", $time);
      #25;
      $display("[%0t] RST: reset released", $time);
      -> reset_done;
    endtask
  endclass

  class alu_driver;
    event reset_done;                    // will refer to the reset agent's event

    task run();
      wait (reset_done.triggered);
      $display("[%0t] DRV: reset finished, start driving", $time);
    endtask
  endclass

  class alu_monitor;
    event reset_done;

    function new(event reset_done);      // alternative: receive the event at construction
      this.reset_done = reset_done;
    endfunction

    task run();
      wait (reset_done.triggered);
      $display("[%0t] MON: reset finished, start sampling", $time);
    endtask
  endclass
```

```
  reset_agent rst;
  alu_driver  drv;
  alu_monitor mon;

  initial begin
    rst = new();
    drv = new();
    drv.reset_done = rst.reset_done;      // method 1: assign the handle
    mon = new(rst.reset_done);            // method 2: pass it to the constructor

    fork                                  // only now start the threads
      rst.run();
      drv.run();
      mon.run();
    join
    $display("[%0t] TOP: all components finished", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] RST: asserting reset
[25] RST: reset released
[25] DRV: reset finished, start driving
[25] MON: reset finished, start sampling
[25] TOP: all components finished
```

One trigger wakes both waiters. The standard does not define which of the two runs first, so the `DRV` and `MON` lines may appear in the opposite order on your simulator. The `TOP` line is always last, because `join` waits for all three tasks.

Line by line

Line	Explanation
`event reset_done;` in each class	Each object initially gets its own event object.
`drv.reset_done = rst.reset_done;`	The driver's handle now refers to the reset agent's event object. The driver's original event object is discarded.
`mon = new(rst.reset_done);`	The same result through a constructor argument.
`fork ... join`	The three `run()` tasks become three threads. They are started only after the handles are connected (see the merging hazard in Section 2.4).
`wait (reset_done.triggered);`	Level-sensitive wait. It is the safe choice for a one-shot notification: it works even if a component happens to reach this line in the same time step in which reset was released.
`-> reset_done;`	Executed by the reset agent at time 25; wakes the driver and the monitor.

What events cannot do

Limitation	Use instead
An event carries no data	Mailbox
An event is not remembered across time steps	A flag variable, a semaphore key, or a mailbox message
An event does not count: two triggers before the waiter runs look like one	Mailbox, or a counter

Limitation	Use instead
An event gives no mutual exclusion	Semaphore

> **Verification application:** The standard generator/driver handshake uses an event in the reverse direction to the data: the generator puts a transaction in the mailbox and then waits for `drv_done`; the driver triggers `drv_done` when it has finished driving the transaction. Part 4 shows the complete code.

Interview questions

Q1. How do you share an event between two classes? Declare an event property in both classes and make both handles refer to the same event object, either by assigning one to the other (`drv.done = gen.done;`), or by passing the event as a constructor or task argument. Then one object triggers it and the other waits on it.

Q2. Why must the event handles be connected before the `run()` tasks are forked? A thread that is already waiting on an event stays attached to the object the handle referred to at that time. If the handle is reassigned afterwards, the thread waits on an object that nobody can trigger any more.

Q3. Can an event pass data from one thread to another? No. It only signals that something occurred. Data is passed through a mailbox, or through a shared object whose handle both threads hold.

2.6 wait_order

In plain English

`wait_order` waits for several events and checks that they occur in a given sequence. It is a small built-in protocol checker: "first a request, then a grant, then done; anything else is an error".

Technical definition

`wait_order (ev1, ev2, ..., evN)` suspends the calling process until all the listed events have been triggered **in the listed order**. If an event is triggered out of order (a later event in the list fires while an earlier one is still awaited), the construct **fails**.

Form	On success	On failure
`wait_order (a, b, c);`	The process continues	A run-time error is reported
`wait_order (a, b, c) else stmt;`	The process continues	`stmt` is executed, no error
`wait_order (a, b, c) pass_stmt; else fail_stmt;`	`pass_stmt` is executed	`fail_stmt` is executed

Rules

- The events must be triggered in order, from left to right, for the construct to succeed.
- An event that has already been matched may trigger again without causing a failure. What is not allowed is a *later* event of the list triggering before its turn.
- The failure is detected at the moment the out-of-order event is triggered. The process does not wait for the rest of the sequence.

Complete example

tb_wait_order.sv

```
module tb_wait_order;

  event req, gnt, done;

  initial begin : stimulus
    #10 -> req;        // time 10
    #10 -> gnt;        // time 20
    #10 -> done;       // time 30 : req, gnt, done  -> correct order

    #10 -> gnt;        // time 40 : gnt before req  -> wrong order
    #10 -> req;        // time 50
    #10 -> done;       // time 60
  end

  initial begin : order_check
    wait_order (req, gnt, done)
      $display("[%0t] PASS: req -> gnt -> done", $time);
    else
      $display("[%0t] FAIL: events out of order", $time);

    #5;                // start the second check at time 35

    wait_order (req, gnt, done)
      $display("[%0t] PASS: req -> gnt -> done", $time);
    else
      $display("[%0t] FAIL: events out of order", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[30] PASS: req -> gnt -> done
[40] FAIL: events out of order
```

Line by line

Line	Explanation
First `wait_order`	Starts at time 0. `req` at 10, `gnt` at 20 and `done` at 30 arrive in order. The pass statement runs at time 30.
`#5;`	The second check starts at time 35, clear of the triggers at time 30.
Second `wait_order`	It expects `req` first. At time 40 `gnt` is triggered instead. The sequence can no longer be satisfied, so the `else` statement runs at time 40. The later triggers at 50 and 60 are not examined.

Common mistake: Using `wait_order` without an `else` clause in a testbench that is supposed to keep running after a protocol error. Without `else`, a failure is a run-time error issued by the simulator, in the simulator's own format, which regression scripts may not recognise. Provide an `else` with your own `$error` message.

Verification application: `wait_order` is occasionally used in a monitor or checker to confirm that the phases of a simple protocol occur in sequence. For anything involving clocks and signal values, concurrent assertions (Module 10) are the normal tool; `wait_order` only works on named events.

Interview questions

Q1. What does `wait_order` do? It blocks the process until a list of events has been triggered in the specified order. If the events occur out of order it fails: it reports a run-time error, or executes the `else` statement if one is given.

Q2. Write the statement that waits for events `a`, `b`, `c` in order and prints a message if they occur out of order. `wait_order (a, b, c) else $display("out of order");`

Q3. What is the difference between `wait_order (a, b, c);` and `@(a); @(b); @(c);`? The sequence of three `@` statements also waits for the events one after another, but it cannot detect a violation: if `b` triggers before `a`, it is simply missed, and the code then waits for another `b`. `wait_order` treats the early `b` as a failure and reports it immediately.

2.7 Events: Summary, Mistakes and Questions

Summary table

Item	Syntax	Remember
Declare	`event ev;`	No `new()`. It is a handle
Trigger now	`-> ev;`	Wakes current waiters only; does not block
Trigger in the NBA region	`->> ev;`	Removes the same-time-step race; a delay may be added
Wait, edge-sensitive	`@(ev);`	Must already be waiting when the trigger occurs
Wait, level-sensitive	`wait (ev.triggered);`	True until the end of the trigger's time step
Wait for one of several	`@(ev1 or ev2);`	Either event unblocks
Check order	`wait_order (a, b, c) else ...;`	Fails on an out-of-order trigger
Alias	`b = a;`	Both names refer to one event
Disconnect	`ev = null;`	Trigger has no effect; waiting is undefined
Compare	`a == b`, `ev == null`	Same object?

Common mistakes

Mistake	Effect	Fix
Trigger and `@` in the same time step from different threads	Hang on some simulators	`wait (ev.triggered)` or `->>`
`wait (ev.triggered)` in a zero-time loop	Simulation hangs at one time	Use `@(ev)` in loops
`@(ev.triggered)`	Waits for a change of the property, races	`wait (ev.triggered)`
Assuming an event is remembered	Waiter that arrives late hangs	Use a flag, mailbox or semaphore when state must persist
Reassigning an event handle while a thread waits on it	That thread never wakes	Connect handles before forking threads

Mistake	Effect	Fix
Declaring separate events in two classes and never connecting them	The waiter waits on its own private event forever	Share the handle
Trying to pass data with an event	Not possible	Mailbox

Further interview questions

Q1. Two processes trigger the same event at the same simulation time, and one process is waiting. How many times does the waiter wake up? Normally once. The first trigger makes the waiter ready, and when the second trigger occurs it is no longer on the waiting list, so the second trigger is lost. Only if the waiter is in a loop and the simulator happens to run it between the two triggers can it wake twice. The count therefore cannot be relied on: events do not count triggers. Use a mailbox or a counter when every occurrence matters.

Q2. How would you wait for either of two events, and how for both? Either: `@(ev1 or ev2);`. Both, in any order: start two threads with `fork @(ev1); @(ev2); join`. Both, in a fixed order: `wait_order (ev1, ev2);`.

Q3. Can an event be used in a sensitivity list such as `always @(ev)`? Yes. An `always @(ev)` block executes each time the event is triggered.

Q4. Is `@(posedge ev)` legal? No. An event has no value, so it has no edges. Only `@(ev)` is allowed.

Q5. What is the lifetime of the `triggered` state? From the instant of the trigger to the end of the current time step. It is cleared when simulation time advances.

Q6. How would you give an event a "memory" so that a waiter arriving later still sees it? Pair it with a flag: the triggering thread sets `done = 1;` and triggers the event; the waiting thread executes `wait (done);`. Alternatively use a semaphore created with zero keys (the notifier puts a key, the waiter gets it) or a mailbox message. In UVM, `uvm_event` provides this with `wait_on()`.

Part 3: Fork and Join

3.1 Sequential and Parallel Blocks

In plain English

A `begin` ... `end` block is a to-do list for one worker: the statements are done one after another. A `fork` ... `join` block hires one worker **per statement**: all the statements start together and each proceeds at its own pace. The word after the block (`join`, `join_any` or `join_none`) only decides how long the person who hired the workers waits for them.

Technical definition

Block	Name	Behaviour
`begin` ... `end`	Sequential block	Statements execute in order, as part of the current process
`fork` ... `join` / `join_any` / `join_none`	Parallel block	**Each statement** inside becomes a separate child process. All children are created at the same simulation time

The process that executes the `fork` is the **parent**. The processes it creates are its **children** (also called sub-processes or spawned threads). A child that itself executes a `fork` creates grandchildren, and so on: processes form a tree.

What counts as one thread

Each *statement* directly inside the `fork` is one thread. To put several statements into one thread, group them with `begin` ... `end`.

```
fork
  statement_1;              -> thread 1
  begin                     -> thread 2 (the whole begin...end block, executed in order)
    statement_2a;
    statement_2b;
  end
  task_call();              -> thread 3 (everything the task does)
join
```

Why it is needed

- A testbench must run its components at the same time: `fork gen.run(); drv.run(); mon.run(); scb.run(); join_none`.
- A test must apply stimulus on several interfaces at once.
- Checks need time limits: "wait for the response, but give up after 1000 clock cycles".
- A design with several independent interfaces needs traffic on all of them concurrently to find the bugs that occur only when things overlap.

`initial` and `always` blocks also create threads, but only fixed ones, at the module level. Classes cannot contain `initial` or `always` blocks. `fork` is the only way for class-based code to start a thread, and it can create any number of threads at run time.

Complete example

tb_fork_threads.sv

```
module tb_fork_threads;

  initial begin
    fork
      #10 $display("[%0t] thread 1 (a single statement)", $time);

      begin
        #5  $display("[%0t] thread 2: first statement", $time);
        #20 $display("[%0t] thread 2: second statement", $time);
      end

      #15 $display("[%0t] thread 3 (a single statement)", $time);
    join
    $display("[%0t] parent: all threads finished", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[5] thread 2: first statement
[10] thread 1 (a single statement)
[15] thread 3 (a single statement)
[25] thread 2: second statement
[25] parent: all threads finished
```

Line by line

Line	Explanation
`fork`	Three child threads are created at time 0.
`#10 $display(...)`	Thread 1 is one statement: wait 10, print.
`begin ... end`	Thread 2 is the whole block. Its two statements run **sequentially** inside the thread: the second delay starts when the first statement is done, so it prints at 5 and at 5 + 20 = 25.
`#15 $display(...)`	Thread 3 prints at 15. All three delays are counted from time 0, because the threads started together.
`join`	The parent waits until the last thread (thread 2) finishes at 25.

> **Common mistake:** Forgetting `begin` ... `end`. In `fork drive_header(); drive_payload(); join` the header and the payload are driven **at the same time** on the same pins. If they must follow each other they belong in one thread: `fork begin drive_header(); drive_payload(); end ... join`.

Interview questions

Q1. What is the difference between `begin`...`end` and `fork`...`join`? Statements in `begin`...`end` execute sequentially in one process. Each statement in `fork`...`join` executes as its own process, and all of them start at the same simulation time.

Q2. How many threads does this create: `fork a(); begin b(); c(); end d(); join`? Three: `a()`, the block containing `b()` then `c()`, and `d()`.

Q3. Is `fork ... join` synthesizable? No. It is a simulation construct used in testbenches and behavioural models.

3.2 fork ... join

Definition

`fork` ... `join` creates the child threads and **blocks the parent until every child has finished**. It is the only form that existed in Verilog.

Syntax

```
fork
  thread_1;
  thread_2;
  thread_3;
join
// reached only when thread_1, thread_2 and thread_3 have all completed
```

Complete example

`tb_fork_join.sv`

```
module tb_fork_join;

  initial begin
    $display("[%0t] parent: before fork", $time);
    fork
      begin #30 $display("[%0t] thread A done", $time); end
      begin #10 $display("[%0t] thread B done", $time); end
      begin #20 $display("[%0t] thread C done", $time); end
    join
    $display("[%0t] parent: after join", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] parent: before fork
[10] thread B done
[20] thread C done
[30] thread A done
[30] parent: after join
```

The children finish in order of their delays, not in the order they are written. The parent resumes at time 30, when the slowest child (A) has finished.

When to use it

Use	Example
Do several things in parallel and continue only when all are complete	Drive a write on port 0 and on port 1 at the same time; then check both

Use	Example
Start reset and clock-stabilisation activities together	`fork apply_reset(); wait_pll_lock(); join`
Run several directed sequences concurrently	`fork seq_a.run(); seq_b.run(); join`

> **Interview trap:** If any child of a `fork` ... `join` never finishes (for example it contains a `forever` loop, as most `run()` tasks of drivers and monitors do), the `join` never completes and the code after it is never reached. This is why the components of an environment are started with `join_none` or `join_any`, not `join`.

3.3 fork ... join_any

Definition

`fork` ... `join_any` creates the child threads and **blocks the parent until any one child has finished**. The remaining children are **not** stopped: they keep running in the background.

Complete example

`tb_fork_join_any.sv`

```
module tb_fork_join_any;

  initial begin
    $display("[%0t] parent: before fork", $time);
    fork
      begin #30 $display("[%0t] thread A done", $time); end
      begin #10 $display("[%0t] thread B done", $time); end
      begin #20 $display("[%0t] thread C done", $time); end
    join_any
    $display("[%0t] parent: after join_any", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] parent: before fork
[10] thread B done
[10] parent: after join_any
[20] thread C done
[30] thread A done
```

The parent resumes at time 10, as soon as the fastest child (B) finishes. Threads C and A continue and print at 20 and 30, even though the parent's `initial` block has already ended.

When to use it

Use	Example
Timeout: wait for an operation, but only up to a limit	One thread waits for the response, another waits for the time limit; whichever finishes first ends the wait
First of several: continue when any one of several conditions occurs	Wait for `done` **or** `error`

Use	Example
End a test when the stimulus generator is finished while the driver and monitor loop forever	`fork gen.run(); drv.run(); mon.run(); join_any`

> **Common mistake:** Forgetting that the other threads are still alive after `join_any`. In a timeout structure the losing thread must be stopped with `disable fork` (Section 3.8). Otherwise the "timeout" thread fires later and reports a timeout for an operation that completed long ago.

3.4 fork ... join_none

Definition

`fork` ... `join_none` creates the child threads and **does not block the parent at all**. The parent continues with the statement after `join_none` immediately.

There is one more rule, and it is asked in interviews: **the child threads do not start executing until the parent thread blocks or terminates.** At the `join_none` the children are only *scheduled*. The parent keeps the processor until it reaches a blocking statement.

Complete example

tb_fork_join_none.sv

```
module tb_fork_join_none;

  initial begin
    $display("[%0t] parent: before fork", $time);
    fork
      $display("[%0t] thread D (no delay) runs", $time);
      begin #30 $display("[%0t] thread A done", $time); end
      begin #10 $display("[%0t] thread B done", $time); end
      begin #20 $display("[%0t] thread C done", $time); end
    join_none
    $display("[%0t] parent: after join_none", $time);
    #5;
    $display("[%0t] parent: continues its own work", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] parent: before fork
[0] parent: after join_none
[0] thread D (no delay) runs
[5] parent: continues its own work
[10] thread B done
[20] thread C done
[30] thread A done
```

Line by line

Line	Explanation
`join_none`	Four children are scheduled. None of them has run yet.

Line	Explanation
`parent: after join_none`	Printed **before** thread D's message although thread D has no delay. The parent has not blocked yet, so no child can run.
`#5;`	The parent blocks. Now the children start, at time 0. Thread D prints immediately; A, B and C begin their delays.
`parent: continues its own work`	The parent and the children now run independently.

When to use it

Use	Example
Start background activity that runs for the whole test	Clock generators in classes, monitors, scoreboards, protocol checkers
Start all components of an environment and return control to the test	`fork gen.run(); drv.run(); mon.run(); scb.run(); join_none`
Launch a variable number of threads from a loop, then collect them with `wait fork`	One thread per port, per channel, per agent

> **Interview trap:** "When do the threads of a `fork` ... `join_none` begin to execute?" Not at the `fork`, and not at the `join_none`. They begin when the parent executes a blocking statement (or ends). Any statements between `join_none` and that blocking statement are executed first.

3.5 Differences Between join, join_any and join_none

Timeline

All three examples above used the same three children: B takes 10 time units, C takes 20, A takes 30.

```
time         0          10          20          30
             |-----------|-----------|-----------|
thread B     ===========>|
thread C     ======================>|
thread A     ==================================>|

join         parent blocked ......................  resumes at 30   (all children done)
join_any     parent blocked .....  resumes at 10                   (first child done)
join_none    resumes at 0                                         (no waiting)

In all three cases every child runs to completion unless it is explicitly disabled.
```

Comparison table

Property	`join`	`join_any`	`join_none`
Parent resumes when	**All** children have finished	**Any one** child has finished	**Immediately**
Parent blocked at the join	Yes	Yes	No

Property	`join`	`join_any`	`join_none`
Children still running when the parent resumes	None	All except the one that finished	All
When children start executing	When the parent blocks at `join`	When the parent blocks at `join_any`	When the parent next blocks or terminates
Remaining children are killed automatically	Not applicable	**No**	**No**
Available in Verilog	Yes	No	No
Allowed inside a function	No	No	Yes
Typical use	Parallel actions that must all complete	Timeouts, "first of"	Background threads, starting components
Usually followed by	Nothing	`disable fork`	`wait fork` (when the results are needed)

The same structure with each join

```
// 1. Both operations must finish before the check
fork
  write_port(0);
  write_port(1);
join
check_both();

// 2. Wait for the response, with a time limit
fork
  wait_for_response();
  #1000;
join_any
disable fork;

// 3. Start the components; the test continues at once
fork
  drv.run();
  mon.run();
join_none
```

Interview questions

Q1. What is the difference between `join`, `join_any` and `join_none`? All three start every thread of the block in parallel. `join` blocks the parent until all threads finish. `join_any` blocks the parent until the first thread finishes; the others keep running. `join_none` does not block the parent; the threads start when the parent next blocks.

Q2. After `join_any`, what happens to the threads that have not finished? They continue to run. They must be terminated explicitly with `disable fork` (or by disabling a named block, or through a `process` handle) if they are no longer wanted.

Q3. Three threads have delays of 5, 10 and 15. After which simulation time does the statement following the block execute for `join`, `join_any` and `join_none`? 15, 5 and 0.

Q4. Which form of fork can be used inside a function, and why? Only `fork` ... `join_none`. A function must not consume simulation time. `join` and `join_any` would block the function; `join_none` returns immediately and the spawned threads run independently.

Q5. In an environment's `run()` task, why are the components started with `join_none` or `join_any` and not `join`? Because drivers, monitors and scoreboards normally contain `forever` loops and never finish. A `join` would wait for them forever and the test could never end.

3.6 Variables in Forked Threads and the Loop Trap

How children see variables

A child thread can read and write the variables that are visible where the `fork` is written: module variables, class properties, and the local variables of the enclosing task or block. It does **not** get a private copy. All children, and the parent, share the same variable.

The classic bug

The task: start three threads, numbered 0, 1 and 2.

`tb_fork_loop_bug.sv`

```
module tb_fork_loop_bug;

  initial begin
    for (int i = 0; i < 3; i++) begin
      fork
        begin
          #(10 * (i + 1));
          $display("[%0t] thread %0d finished", $time, i);
        end
      join_none
    end
    wait fork;
    $display("[%0t] parent: all threads finished", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[40] thread 3 finished
[40] thread 3 finished
[40] thread 3 finished
[40] parent: all threads finished
```

Every thread reports number 3, a value the loop body was never supposed to see, and all of them finish at time 40.

Why

```
parent                                              children
------                                              --------
i = 0 : fork ... join_none   -> child 1 scheduled   (not running yet)
i = 1 : fork ... join_none   -> child 2 scheduled   (not running yet)
i = 2 : fork ... join_none   -> child 3 scheduled   (not running yet)
i = 3 : loop condition false, loop ends
wait fork;  <-- the parent blocks here for the first time
                                                    child 1 starts: reads i  -> 3
                                                    child 2 starts: reads i  -> 3
                                                    child 3 starts: reads i  -> 3
```

1. With `join_none`, no child runs until the parent blocks (Section 3.4).

2. The parent does not block inside the loop, so the loop runs to completion first and `i` ends at 3.
3. All three children share the single variable `i`. When they finally start, each reads 3: the delay is `10 * (3 + 1)` = 40 and the printed number is 3.

The fix

Give each thread its own copy of the loop value, in a variable that is created anew for every iteration: an **automatic** variable declared inside the loop body.

`tb_fork_loop_fixed.sv`

```
module tb_fork_loop_fixed;

  initial begin
    for (int i = 0; i < 3; i++) begin
      automatic int id = i;            // a new variable for each iteration, copied NOW
      fork
        begin
          #(10 * (id + 1));
          $display("[%0t] thread %0d finished", $time, id);
        end
      join_none
    end
    wait fork;
    $display("[%0t] parent: all threads finished", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[10] thread 0 finished
[20] thread 1 finished
[30] thread 2 finished
[30] parent: all threads finished
```

Line by line

Line	Explanation
`automatic int id = i;`	Executed by the **parent** in each iteration, while `i` still has the value of that iteration. `automatic` means a fresh `id` is allocated every time the block is entered, so there are three separate variables holding 0, 1 and 2.
The child's use of `id`	Each child refers to the `id` of its own iteration. The variable stays alive as long as a thread still refers to it, even though the loop body has finished.
`wait fork;`	Blocks the parent until all three children have completed (Section 3.7).

An equivalent form declares the variable inside the `fork` block. Declarations at the top of a `fork` block are initialised by the parent when the `fork` is executed, before any child starts:

```
for (int i = 0; i < 3; i++)
  fork
    automatic int id = i;              // initialised at the fork, by the parent
    begin
      #(10 * (id + 1));
      $display("[%0t] thread %0d finished", $time, id);
    end
  join_none
```

When the keyword automatic is required

Where the loop is written	Lifetime of a local variable by default	What to write
`initial` / `always` block in a module or interface, static task	Static: one copy for the whole simulation	`automatic int id = i;` (the keyword is required)
Class method	Automatic	`int id = i;` is enough; `automatic` is harmless
Task or function declared `automatic`, or inside a `program` or module declared `automatic`	Automatic	`int id = i;`

Without `automatic` in a module-level `initial` block, `int id = i;` declares a single static variable; it would be shared by all threads exactly like `i`. Writing `automatic` always is the safe habit.

> **Interview trap:** The wrong fix is to copy the value **inside** the child: `fork begin int id = i; ... end join_none`. The copy is then made when the child starts running, which is after the loop has ended, so it copies 3. The copy must be made by the parent, outside the child's `begin` ... `end`.

> **Verification application:** The same pattern starts one thread per array element: `foreach (drv[i]) begin automatic int k = i; fork drv[k].run(); join_none end`. It is used for multi-port DUTs, where the number of drivers and monitors is a parameter.

Interview questions

Q1. What does this print, and why? `for (int i = 0; i < 3; i++) fork $display(i); join_none` It prints 3 three times. The threads of a `join_none` do not start until the parent blocks, which is after the loop has finished, and they all share the one loop variable, whose final value is 3.

Q2. How do you fix it? Declare an automatic variable inside the loop (or in the declaration part of the `fork` block), initialised from the loop variable, and use that in the thread: `automatic int j = i;`.

Q3. Would the bug occur with `fork` ... `join` instead of `join_none`? No. With `join` the parent blocks in every iteration until the child has finished, so the child reads `i` while it still has the right value. But then the threads run one after another, not in parallel, which defeats the purpose.

Q4. Why does an automatic variable solve the problem when a static one does not? A static variable exists once; every iteration overwrites it and every thread sees the last value. An automatic variable is created each time its block is entered, so each iteration, and therefore each thread, has its own copy.

3.7 wait fork

Definition

`wait fork;` blocks the calling process until **all of its immediate child processes** have finished. "Immediate children" means the threads this process itself created with `fork`; the grandchildren that those threads may have created are not waited for.

Why it is needed

`join_none` lets the parent continue, and a loop of `join_none` blocks creates a number of threads that is not known when the code is written. There is no `join` to wait at. `wait fork` is the way to say "now wait for everything I have started".

Syntax and use

```
for (int i = 0; i < n; i++) begin
  automatic int id = i;
  fork
    send_packet(id);
  join_none
end
wait fork;                    // all n send_packet() threads have completed
```

The example in Section 3.6 (`tb_fork_loop_fixed`) shows it in a complete program: the parent's last message is printed at time 30, when the slowest thread finishes.

Rules

Rule	Consequence
It waits for **all** children of the calling process, not only those of the most recent `fork`	A background thread started earlier with `join_none` in the same process is included. If that thread contains a `forever` loop, `wait fork` never returns
It waits only for immediate children	Threads created by the children may still be running afterwards
If the process has no active children it does not block	It is safe to call when nothing was forked

wait fork versus join

Property	`join`	`wait fork`
Waits for	The threads of that one `fork` block	Every child thread of the current process
Where it is written	At the end of the `fork` block	Anywhere later in the parent
Works with threads created in a loop	Only one iteration at a time (sequentially)	Yes: start all with `join_none`, then wait once

> **Common mistake:** Calling `wait fork` in a `run()` task that has earlier started a monitor or clock thread with `join_none`. The call never returns. The cure is the guard described in the next section: do the forking and the `wait fork` inside a dedicated thread, so that "all my children" means only the intended ones.

Interview questions

Q1. What does `wait fork` do? It suspends the calling process until all the child processes it has spawned have terminated.

Q2. What is the difference between `wait fork` and `fork` ... `join`? `join` waits for the threads of one specific `fork` block. `wait fork` waits for all children of the calling process, whichever `fork` created them. It is used after `join_none` or `join_any`, typically when threads are created in a loop.

Q3. Does `wait fork` wait for the threads spawned by its children? No, only for the immediate children.

3.8 disable fork and disable label

In plain English

After `join_any` the parent has what it wanted from one thread, and the others are still running. `disable fork` stops them. More precisely, it stops **every thread the current process has spawned, and everything those threads have spawned**. That reach is larger than most people expect, and is the source of a well-known bug.

Technical definition

Statement	What it terminates
`disable fork;`	All active **descendants** (children, grandchildren, ...) of the **calling process**. It is not limited to the nearest `fork` block
`disable block_name;`	The named `begin` ... `end` or `fork` ... `join` block, and everything started inside it. Execution continues after the block
`disable task_name;`	The named task. All currently active executions of it are terminated

Complete example: a timeout

```
fork
  begin
    wait_for_response();                // thread 1: the operation
    $display("response received");
  end
  begin
    #1000;                              // thread 2: the time limit
    $display("TIMEOUT");
  end
join_any                                // continue when either finishes
disable fork;                           // stop the one that has not finished
```

The bug: disable fork kills too much

tb_disable_fork_bug.sv

```
module tb_disable_fork_bug;

  initial begin
    fork                                          // a background thread
      forever #10 $display("[%0t] heartbeat", $time);
    join_none

    fork                                          // a timeout structure
      begin #25 $display("[%0t] response received", $time); end
      begin #15 $display("[%0t] timeout", $time); end
    join_any
    disable fork;                                 // intended: stop the slower of the two
    $display("[%0t] disable fork executed", $time);

    #30;
    $display("[%0t] end of test", $time);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[10] heartbeat
[15] timeout
[15] disable fork executed
[45] end of test
```

The heartbeat stops after time 10. `disable fork` terminated the response thread, as intended, **and the heartbeat thread**, because the heartbeat is also a child of the same `initial` process. In a real testbench the "heartbeat" is a monitor, a scoreboard or a clock generator, and the symptom is that checking silently stops halfway through the test.

The fix: a guard thread

Put the timeout structure inside its own thread. Inside that thread, "all my descendants" means only the two threads of the timeout structure.

tb_disable_fork_guard.sv

```
module tb_disable_fork_guard;

  initial begin
    fork                                              // a background thread
      forever #10 $display("[%0t] heartbeat", $time);
    join_none

    fork                                              // guard: fork begin ... end join
      begin
        fork
          begin #25 $display("[%0t] response received", $time); end
          begin #15 $display("[%0t] timeout", $time); end
        join_any
        disable fork;                                 // kills only the guard's children
      end
    join
    $display("[%0t] disable fork executed", $time);

    #30;
    $display("[%0t] end of test", $time);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[10] heartbeat
[15] timeout
[15] disable fork executed
[20] heartbeat
[30] heartbeat
[40] heartbeat
[45] end of test
```

```
   Process tree in the buggy version            Process tree in the guarded version

   initial                                      initial
     |-- heartbeat        <- killed               |-- heartbeat            <- survives
     |-- response thread  <- killed               |-- guard thread
     |-- timeout thread   (already finished)            |-- response thread  <- killed
                                                        |-- timeout thread
   disable fork is executed by "initial":
   everything below it dies.                    disable fork is executed by "guard thread":
                                                only what is below the guard dies.
```

The outer `fork begin ... end join` creates exactly one child and waits for it, so it adds no parallelism. Its only purpose is to create a new process to act as the owner of the inner threads.

> **Key point:** The idiom `fork begin ... fork ... join_any disable fork; end join` is the standard, safe way to write a timeout or a "first of" construct. Use it whenever the surrounding code may have other threads running, which in a class-based testbench is always.

disable with a block name

```
fork : watch
  begin #25 $display("response received"); end
  begin #15 $display("timeout");           end
join_any
disable watch;                // terminates the threads of the block named "watch" only
```

A named `disable` does not touch unrelated threads, so it avoids the bug above without a guard. It has its own trap:

> **Interview trap:** `disable block_name` (and `disable task_name`) acts on the block or task **by name**, not on one particular execution of it. If the same task is running concurrently in several threads, or the named block is inside a class method that is running in several objects, the `disable` terminates the block in **all** of them. `disable fork` affects only the descendants of the calling process, which is why it is preferred in class-based code.

Comparison

Property	`disable fork`	`disable name`	`process.kill()`
What is terminated	All descendants of the calling process	The named block or task, all active executions	The one process the handle refers to, and its descendants
Selective	No (everything below the caller)	By name	Yes, exactly one thread
Needs a name or handle	No	A block or task name	A `process` handle
Main risk	Kills unrelated background threads	Kills other executions of the same named code	None, but more code to write
Covered in	This section	This section	Module 11

Interview questions

Q1. What does `disable fork` do? It terminates all active threads that were spawned by the calling process, including their own child threads.

Q2. Does `disable fork` kill only the threads of the nearest `fork` block? No. It kills every descendant of the process that executes it. Threads started earlier by the same process, for example with `join_none`, are killed as well.

Q3. How do you make `disable fork` affect only one `fork` block? Enclose the block and the `disable fork` in a guard thread: `fork begin fork ... join_any disable fork; end join`. The `disable fork` is then executed by the guard thread, whose only descendants are the threads of the inner block.

Q4. What is the difference between `disable fork` and `disable <label>`? `disable fork` terminates the descendants of the current process and needs no name. `disable <label>` terminates a specific named block or task wherever it is executing, which can include other threads or objects running the same code.

Q5. Write code that waits for an event `done` but gives up after 500 time units.

```
fork
  begin
    fork
      @(done);
      #500 $error("timeout waiting for done");
    join_any
    disable fork;
  end
join
```

3.9 Nested fork-join

Definition

A thread created by a `fork` may itself contain a `fork`. The inner block's join keyword controls when **that thread** continues; the outer block's join keyword controls when the **parent** continues. Each join only looks at the threads of its own block.

Complete example

tb_fork_nested.sv

```
module tb_fork_nested;

  initial begin
    fork
      begin : branch_1
        fork
          #10 $display("[%0t] 1a done", $time);
          #20 $display("[%0t] 1b done", $time);
        join
        $display("[%0t] branch 1 complete (waited for 1a and 1b)", $time);
      end

      begin : branch_2
        #5;
        fork
          #10 $display("[%0t] 2a done", $time);
          #30 $display("[%0t] 2b done", $time);
        join_any
        $display("[%0t] branch 2 complete (waited for 2a only)", $time);
      end
    join
    $display("[%0t] parent: outer join complete", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[10] 1a done
[15] 2a done
[15] branch 2 complete (waited for 2a only)
[20] 1b done
[20] branch 1 complete (waited for 1a and 1b)
[20] parent: outer join complete
[35] 2b done
```

```
   Process tree                          Timeline

   initial (parent)                      time   0    5   10   15   20        35
     |-- branch_1                                |----|----|----|----|---------|
     |     |-- 1a                        1a     =========>
     |     |-- 1b                        1b     ==================>
     |-- branch_2                        branch_1 ends .............^ (join: 1a and 1b)
           |-- 2a                        2a           ==========>
           |-- 2b                        2b           ============================>
                                         branch_2 ends ........^ (join_any: 2a)
                                         outer join ...............^ (both branches)
```

Line by line

Time	Explanation
0	The outer `fork` creates `branch_1` and `branch_2`. `branch_1` immediately forks `1a` and `1b` and blocks at its inner `join`. `branch_2` waits 5.
5	`branch_2` forks `2a` (finishes at 15) and `2b` (finishes at 35) and blocks at its inner `join_any`.
10	`1a` finishes.
15	`2a` finishes. The inner `join_any` is satisfied, so `branch_2` prints and ends. `2b` is still running.
20	`1b` finishes. The inner `join` is satisfied, so `branch_1` prints and ends. Both children of the outer `fork` have now ended, so the outer `join` completes.
35	`2b` finishes. It is a grandchild; the outer `join` did not wait for it.

> **Key point:** A `join` waits for the threads of its own block, which are its immediate children. It does not wait for threads that those children left running with `join_any` or `join_none`.

Interview questions

Q1. In a nested fork, does the outer `join` wait for threads created by an inner `join_none`? No. The outer `join` waits until each of its own threads has finished executing its statements. A thread that executes an inner `fork` ... `join_none` finishes without waiting for the inner threads, which carry on independently.

Q2. An inner thread is still running when its parent thread ends. What happens to it? It continues to run. A child is not terminated when its parent finishes; it stops only when it completes, when it is disabled, or when the simulation ends.

3.10 Parallel Process Execution and Synchronization Patterns

The common patterns

Requirement	Pattern
Run A and B together; continue when **both** are done	`fork A; B; join`
Continue when **either** is done, stop the other	`fork begin fork A; B; join_any disable fork; end join`
Run A with a time limit	`fork begin fork A; #limit; join_any disable fork; end join`
Start background threads and carry on	`fork A; B; join_none`
Start N threads from a loop and wait for all	loop of `fork ... join_none` with an automatic copy of the index, then `wait fork`
Start N threads and continue when the first finishes	loop of `fork ... join_none`, then wait for an event or flag set by the finishing thread
Start the components of an environment	`fork gen.run(); drv.run(); mon.run(); scb.run(); join_none` and then wait for an end-of-test condition

How an environment uses them

```
task run();
  fork
    gen.run();              // ends after the last transaction
    drv.run();              // forever loop
    mon.run();              // forever loop
    scb.run();              // forever loop
  join_any                  // continue when the generator has finished
  wait (scb.num_checked == gen.num_txn);    // let the last transactions drain
endtask
```

`join_any` returns when the generator, the only thread that ever finishes, is done. The other three keep running so that the transactions still in flight are driven, monitored and checked. Module 9 develops this into the complete environment.

Other rules worth knowing

Rule	Detail
A child outlives the task that created it	If a task executes `fork ... join_none` and returns, the threads keep running
Automatic variables stay alive	Local variables of the parent that a child refers to exist as long as the child does
`return` inside a fork block	Illegal. A thread cannot return from the enclosing task or function
`fork` in a function	Only `join_none` is permitted
End of simulation	`$finish` ends everything. In a `program` block, the simulation also ends when the program's `initial` blocks have all finished, even if threads they forked are still running, so background threads are cut off. Waiting with `wait fork` or an end-of-test event avoids this
Fine-grained control	The built-in `process` class gives a handle to a thread: `process::self()`, `status()`, `kill()`, `await()`, `suspend()`, `resume()`. Covered in Module 11

Common mistakes with fork

Mistake	Effect	Fix
`fork` ... `join` around `forever` threads	Code after `join` never runs	`join_any` or `join_none`
No `disable fork` after `join_any` in a timeout	A stale timeout fires later	`disable fork` inside a guard
`disable fork` without a guard	Background threads die	`fork begin ... end join` guard
Loop variable used directly in a `join_none` thread	Every thread sees the final value	`automatic` copy per iteration
Statements that must be sequential written as separate fork statements	They run at the same time	Group with `begin` ... `end`
Expecting `join_none` threads to start at once	Parent code runs first	Remember that children start when the parent blocks
`wait fork` with a never-ending background child	Hangs	Guard thread, or do not use `wait fork` there

Interview questions

Q1. How do you start an unknown number of threads and wait for all of them? In a loop, make an automatic copy of the loop index and start each thread with `fork` ... `join_none`. After the loop call `wait fork`.

Q2. How do you implement a timeout in SystemVerilog? Start the operation and a delay as two threads of a `fork` ... `join_any`, and after it execute `disable fork` to stop whichever is still running. Wrap the whole construct in `fork begin ... end join` so that the `disable fork` cannot kill unrelated threads.

Q3. What happens to forked threads when the parent task returns? They continue running. Threads are terminated only by finishing, by being disabled or killed, or by the end of simulation.

Q4. How do you kill one specific thread among several? Obtain its `process` handle inside the thread with `process::self()` and call `kill()` on that handle, or give the thread's block a name and use `disable name` (keeping in mind that this disables every active execution of that block).

Q5. A test ends before the scoreboard has checked the last transactions. What is the likely cause? The test finished (called `$finish` or returned) as soon as the generator was done, typically right after a `join_any`, while the driver, monitor and scoreboard still had transactions in flight. The test must wait for an end-of-test condition such as "number checked equals number generated", or for a drain time.

Part 4: Mailbox

4.1 What a Mailbox Is and Why It Is Required

Start with the problem

The generator creates transactions. The driver turns each transaction into pin activity. They are two objects of two classes, running as two threads, and they work at different speeds: the generator can create a transaction in zero simulation time, while the driver needs at least one clock cycle to drive it.

The generator therefore needs to **hand over** each transaction, and the hand-over must cope with the difference in speed:

- If the driver is busy, the transaction must be **kept** until the driver is ready. Nothing may be lost or overwritten (remember `tb_no_ipc` in Section 1.2).
- If the driver is ready and no transaction exists yet, the driver must **wait** without wasting simulation effort.
- Transactions must arrive **in the order** they were sent.
- Optionally, the generator must be **slowed down** so that it does not create thousands of transactions the driver cannot keep up with.

A mailbox provides exactly this.

In plain English

A mailbox is a letterbox between two people. One person drops letters in. The other takes letters out, oldest first. If the box is empty, the person collecting waits beside it until a letter arrives. If the box has a limited size and is full, the person posting waits until there is room.

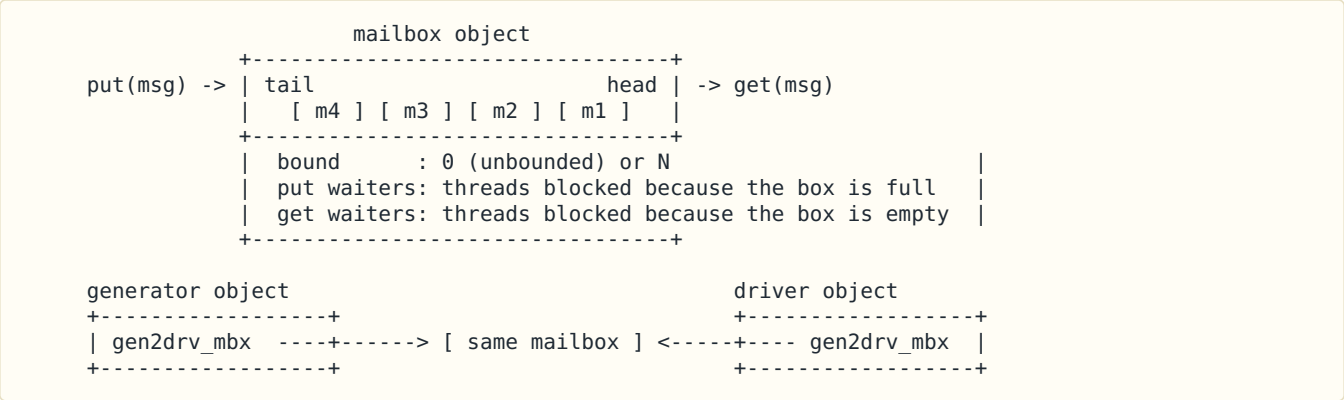
Technical definition

A **mailbox** is a built-in SystemVerilog class that implements a thread-safe **FIFO (first-in, first-out) queue of messages** with built-in blocking:

- `put()` adds a message at the tail. On a full bounded mailbox it blocks.
- `get()` removes the message at the head. On an empty mailbox it blocks.
- A message can be a value of any type: an `int`, a struct, or, most commonly, a **class handle** such as an `alu_txn`.

Because it is a class, a mailbox variable is a **handle**. It is `null` until the mailbox is constructed with `new()`, and several components can hold handles to the same mailbox object, which is how they are connected.

How it works internally



When a `get()` finds the mailbox empty, the calling thread is suspended and added to the list of get waiters. A later `put()` stores the message and makes the first waiter ready. The same happens in the other direction for a full bounded mailbox.

Mailbox versus a shared queue

Property	Queue (`alu_txn q[$]`)	Mailbox (`mailbox #(alu_txn)`)
Stores many items in FIFO order	Yes	Yes
Reader blocks when empty	No. You must write `wait (q.size() > 0);` yourself	Yes, built into `get()` and `peek()`
Writer blocks when full	No. No size limit	Yes, for a bounded mailbox
Safe with several readers and writers	You must add your own protection	Yes
Access to the middle, sorting, searching	Yes	No, only head and tail
Passed between components as	A `ref` argument or a shared object	A handle
Typical use	Storage inside one component (scoreboard expected list)	Connection **between** components

Interview questions

Q1. What is a mailbox? A built-in class that provides a FIFO message queue for passing data between threads. `put()` adds a message and `get()` removes one; `get()` blocks while the mailbox is empty and `put()` blocks while a bounded mailbox is full.

Q2. Why is a mailbox needed between the generator and the driver? They are separate threads that run at different rates. The mailbox stores transactions in order until the driver takes them, makes the driver wait when none is available, and, if bounded, makes the generator wait when the driver is behind. A shared variable would lose data and a plain queue has no blocking.

Q3. Is a mailbox a data type or a class? A class. A mailbox variable is a handle that is `null` until `new()` is called.

Q4. What is the difference between a mailbox and a queue? Both hold items in order. A mailbox adds blocking `get` / `put` and is designed for communication between threads; it gives access only to the head and tail. A queue

is an ordinary data structure with full access and methods for inserting, deleting and searching, but with no synchronization.

4.2 Declaring and Creating a Mailbox: Bounded and Unbounded

Syntax

```
mailbox            mbx_any;            // typeless mailbox handle (can hold any type)
mailbox #(int)     mbx_int;            // parameterized mailbox of int
mailbox #(alu_txn) gen2drv_mbx;        // parameterized mailbox of transaction handles

mbx_int     = new();                   // unbounded: can hold any number of messages
mbx_int     = new(0);                  // the same: a bound of 0 means unbounded
gen2drv_mbx = new(4);                  // bounded: holds at most 4 messages

mailbox #(int) m = new(2);             // declaration and construction together

typedef mailbox #(alu_txn) alu_txn_mbx_t;   // a short name for the type
alu_txn_mbx_t mon2scb_mbx = new();
```

The constructor

`function new(int bound = 0);`

Argument	Result
Omitted, or 0	**Unbounded** mailbox. `put()` never blocks
A positive number N	**Bounded** mailbox with room for N messages. `put()` blocks while N messages are stored
A negative number	Not allowed

Bounded versus unbounded

Property	Unbounded `new()`	Bounded `new(N)`
Capacity	Limited only by memory	N messages
`put()`	Never blocks	Blocks while full
`try_put()`	Always succeeds	Fails (returns 0) while full
`get()`, `peek()`	Block while empty	Block while empty
Flow control of the sender	None: the sender can run arbitrarily far ahead	Built in: the sender is throttled to the receiver's speed
Risk	A fast generator fills memory with thousands of pending transactions; stimulus is decided long before it is used	A deadlock if both sides wait on full/ empty mailboxes in a cycle
Typical use	Monitor → scoreboard (the monitor must never be held up)	Generator → driver when the generator should stay close to the driver

> **Common mistake:** Declaring a mailbox and using it without `new()`: `mailbox #(alu_txn) gen2drv_mbx; gen2drv_mbx.put(txn);` The handle is `null`, and the simulation stops with a null-handle error. The second version of this mistake is constructing **two** mailboxes, one in the generator and one in the driver. Each component then talks to its own private mailbox and nothing ever arrives. One mailbox must be constructed (normally in the environment) and its handle given to both components.

Interview questions

Q1. How do you create a bounded mailbox and an unbounded mailbox? `mbx = new(n);` creates a bounded mailbox holding at most `n` messages. `mbx = new();` or `new(0)` creates an unbounded mailbox.

Q2. What is the default size of a mailbox? Unbounded (bound 0).

Q3. What happens when `put()` is called on a full bounded mailbox? The calling thread blocks until another thread removes a message with `get()` or `try_get()`.

Q4. Can `put()` on an unbounded mailbox ever block? No.

4.3 Mailbox Methods

The complete set

Method	Kind	Blocks	What it does	Return value
`new(int bound = 0)`	Constructor	No	Creates the mailbox	The handle
`num()`	Function	No	Number of messages currently stored	`int`
`put(msg)`	Task	Yes, while full (bounded only)	Stores `msg` at the tail	None
`try_put(msg)`	Function	No	Stores `msg` if there is room	Positive if stored, 0 if full
`get(ref msg)`	Task	Yes, while empty	Removes the head message and copies it into `msg`	None
`try_get(ref msg)`	Function	No	Removes the head message if there is one	Positive if a message was taken, 0 if empty, negative on a type mismatch
`peek(ref msg)`	Task	Yes, while empty	**Copies** the head message into `msg` **without removing it**	None
`try_peek(ref msg)`	Function	No	Copies the head message if there is one, without removing it	Positive if copied, 0 if empty, negative on a type mismatch

Three patterns organise this table:

1. **`put` adds, `get` removes, `peek` looks.**
2. **The plain methods are tasks and may block. The `try_` methods are functions, never block, and report what happened.**

3. Because the `try_` methods are functions, they are the only ones that can be called from a **function** (for example from a scoreboard's `check()` function or from a `final` block).

get versus peek

```
   mailbox before :   tail [ 30 ] [ 20 ] [ 10 ] head

   peek(v)  ->  v = 10      mailbox after :  tail [ 30 ] [ 20 ] [ 10 ] head     (unchanged)
   get(v)   ->  v = 10      mailbox after :  tail [ 30 ] [ 20 ]        head     (10 removed)
```

Property	`get()`	`peek()`
Returns the head message	Yes	Yes
Removes it from the mailbox	Yes	No
Blocks while empty	Yes	Yes
`num()` afterwards	One less	Unchanged
A second call returns	The next message	The same message again
Typical use	Normal consumption	Look at an item before deciding to take it; keep a bounded mailbox full while the item is being processed (Section 4.8)

Complete example

tb_mailbox_methods.sv

```
module tb_mailbox_methods;

  mailbox #(int) mbx;
  int            v;

  initial begin
    mbx = new();                              // unbounded

    mbx.put(10);
    mbx.put(20);
    mbx.put(30);
    $display("1. after three puts : num = %0d", mbx.num());

    mbx.peek(v);
    $display("2. peek    -> %0d    : num = %0d  (message stays)", v, mbx.num());

    mbx.get(v);
    $display("3. get     -> %0d    : num = %0d  (message removed)", v, mbx.num());

    mbx.get(v);
    $display("4. get     -> %0d    : num = %0d", v, mbx.num());

    if (mbx.try_get(v))
      $display("5. try_get -> %0d    : num = %0d", v, mbx.num());

    if (!mbx.try_get(v))
      $display("6. try_get on an empty mailbox returned 0 and did not block");

    if (!mbx.try_peek(v))
      $display("7. try_peek on an empty mailbox returned 0 and did not block");
  end

endmodule
```

EXPECTED OUTPUT

```
1. after three puts : num = 3
2. peek    -> 10    : num = 3  (message stays)
3. get     -> 10    : num = 2  (message removed)
4. get     -> 20    : num = 1
5. try_get -> 30    : num = 0
6. try_get on an empty mailbox returned 0 and did not block
7. try_peek on an empty mailbox returned 0 and did not block
```

Line by line

Line	Explanation
`mbx = new();`	Constructs an unbounded mailbox. Without this line the first `put` would be a null-handle error.
Three `put` calls	The messages are queued in order: 10 at the head, 30 at the tail. None of them blocks.
`mbx.peek(v);`	Copies the head message (10). The mailbox still holds three messages.
`mbx.get(v);`	Returns the same head message (10) and removes it. FIFO order: the first message put is the first one got.
`mbx.try_get(v)`	A message is available (30), so it is taken and the function returns a positive value, which the `if` treats as true.
Second `try_get`	The mailbox is empty. A blocking `get` here would suspend the thread forever, since nobody else puts anything. `try_get` returns 0 and the code continues.

Blocking versus nonblocking methods

	Blocking: `put`, `get`, `peek`	Nonblocking: `try_put`, `try_get`, `try_peek`
Kind	Task	Function
If the operation cannot be done now	The thread waits until it can	Returns 0 immediately
Callable from a function	No	Yes
Callable from a task	Yes	Yes
Consumes simulation time	Possibly	Never
Result must be checked	No	**Yes**: ignoring the return value means not knowing whether anything happened
Typical use	Normal component loops: `forever begin mbx.get(t); ... end`	Polling, draining a mailbox at end of test, code that must not wait

Interview trap: `forever begin if (mbx.try_get(t)) process(t); end` hangs the simulation. When the mailbox is empty, `try_get` returns at once, the loop repeats without any blocking statement, and simulation time can never advance. A polling loop must contain a delay or an event control, for example `@(posedge clk);`. If there is no reason to poll, use the blocking `get`.

Common mistake: Using `num()` to decide whether a following `get()` will block: `if (mbx.num() > 0) mbx.get(t);`. With more than one consumer, another thread can take the message between the test and the `get`. `try_get` performs the test and the removal as one indivisible operation.

Interview questions

Q1. List the mailbox methods. `new()`, `put()`, `try_put()`, `get()`, `try_get()`, `peek()`, `try_peek()`, `num()`.

Q2. What is the difference between `get()` and `peek()`? Both return the message at the head and both block while the mailbox is empty. `get()` removes the message; `peek()` leaves it in the mailbox.

Q3. What is the difference between `get()` and `try_get()`? `get()` is a task that blocks until a message is available. `try_get()` is a function that never blocks: it returns a positive value and the message if one is available, and 0 otherwise.

Q4. What does `try_put()` return? A positive value if the message was stored, and 0 if the bounded mailbox was full.

Q5. Which mailbox methods can be called from a function? `num()`, `try_put()`, `try_get()` and `try_peek()`. The blocking methods are tasks and cannot be called from a function.

Q6. What does `num()` return? The number of messages currently in the mailbox.

4.4 Bounded Mailbox in Action

A mailbox of integer values with flow control

The producer wants to send four values as fast as it can. The mailbox holds two. The consumer takes one value every 10 time units. Only the producer prints, so that the output shows exactly when each `put` was able to complete.

tb_mailbox_bounded.sv

```
module tb_mailbox_bounded;

  mailbox #(int) mbx = new(2);                // bounded: at most 2 messages

  initial begin : producer
    for (int i = 1; i <= 4; i++) begin
      mbx.put(i);                             // blocks while the mailbox is full
      $display("[%0t] producer: put(%0d) completed", $time, i);
    end
  end

  initial begin : consumer
    int v;
    repeat (4) begin
      #10;
      mbx.get(v);                             // frees one place every 10 time units
    end
  end

endmodule
```

EXPECTED OUTPUT

```
[0] producer: put(1) completed
[0] producer: put(2) completed
[10] producer: put(3) completed
[20] producer: put(4) completed
```

Execution flow

Time	Consumer	Producer	Mailbox contents (head first)
0	Waiting for time 10	`put(1)` and `put(2)` complete. `put(3)` blocks: full	1, 2
10	`get` takes 1	A place is free: `put(3)` completes. `put(4)` blocks: full again	2, 3
20	`get` takes 2	`put(4)` completes. The producer has finished	3, 4
30	`get` takes 3		4
40	`get` takes 4		empty

The producer was automatically slowed to the consumer's pace, without any extra code. With an unbounded mailbox all four `put` calls would have completed at time 0.

try_put on a bounded mailbox

tb_mailbox_try_put.sv

```
module tb_mailbox_try_put;

  mailbox #(int) mbx = new(2);

  initial begin
    if (mbx.try_put(1))  $display("1. try_put(1) stored    : num = %0d", mbx.num());
    if (mbx.try_put(2))  $display("2. try_put(2) stored    : num = %0d", mbx.num());
    if (!mbx.try_put(3)) $display("3. try_put(3) refused   : mailbox full, num = %0d",
                                  mbx.num());
  end

endmodule
```

EXPECTED OUTPUT

```
1. try_put(1) stored    : num = 1
2. try_put(2) stored    : num = 2
3. try_put(3) refused   : mailbox full, num = 2
```

The third `try_put` does not block and does not store anything; the value 3 is simply not sent, and it is the caller's job to notice the return value of 0.

Interview questions

Q1. A bounded mailbox of size 1 connects a generator and a driver. The generator calls `put` three times in a row. When does each `put` return? The first returns immediately. The second returns when the driver has removed the first message. The third returns when the driver has removed the second.

Q2. How can a sender avoid blocking on a full bounded mailbox? Use `try_put()` and check its return value.

Q3. Why would you bound the generator-to-driver mailbox? So that the generator cannot run far ahead of the driver. This keeps memory use small, and, more importantly, keeps the generation of each transaction close in time to its use, so that the generator can react to the current state of the testbench (for example to responses or to configuration changes).

4.5 Parameterized and Typeless Mailboxes

Two kinds of mailbox

Declaration	Name	What it can hold	Type checking
`mailbox mbx;`	Typeless (generic, dynamic) mailbox	Messages of **any** type, even mixed in one mailbox	At **run time**, when a message is taken out
`mailbox #(T) mbx;`	Parameterized (typed) mailbox	Only messages of type `T`	At **compile time**

The problem with a typeless mailbox

The receiver must take each message into a variable of exactly the type that was sent. The compiler cannot check this, because any thread may have put anything.

tb_mailbox_typeless.sv

```
module tb_mailbox_typeless;

  mailbox mbx = new();                  // typeless
  int     i_val = 42;
  string  s_val = "hello";
  int     i_out;
  string  s_out;
  int     rc;

  initial begin
    mbx.put(i_val);                     // first message  : an int
    mbx.put(s_val);                     // second message : a string

    mbx.get(i_out);                     // int message into an int variable: fine
    $display("1. got int    : %0d", i_out);

    rc = mbx.try_get(i_out);            // string message into an int variable
    if (rc < 0)
      $display("2. try_get returned a negative value: type mismatch, nothing taken");

    mbx.get(s_out);                     // the string is still there
    $display("3. got string : %s", s_out);

    // mbx.put(s_val);
    // mbx.get(i_out);                  // blocking get with the wrong type: RUN-TIME ERROR
  end

endmodule
```

EXPECTED OUTPUT

```
1. got int    : 42
2. try_get returned a negative value: type mismatch, nothing taken
3. got string : hello
```

Line by line

Line	Explanation
`mailbox mbx = new();`	No type parameter. The compiler accepts any `put` and any `get`.
`mbx.try_get(i_out)` on a string message	The types do not match. `try_get` reports this by returning a negative value and leaves the message in the mailbox.

Line	Explanation
The commented lines	A blocking `get()` or `peek()` into a variable of the wrong type is a **run-time error**. The simulation stops, possibly hours into a regression.

The parameterized mailbox

```
mailbox #(int) mbx = new();
string s = "hello";

mbx.put(5);         // fine
// mbx.put(s);      // COMPILE-TIME error: a string is not an int
```

With the type fixed in the declaration, a wrong `put` or `get` is found by the compiler in seconds. A parameterized mailbox is used in practically all testbench code:

```
typedef mailbox #(alu_txn) alu_txn_mbx_t;      // define the type once

alu_txn_mbx_t gen2drv_mbx;                     // generator -> driver
alu_txn_mbx_t mon2scb_mbx;                     // monitor   -> scoreboard
```

Comparison

Property	Typeless `mailbox`	Parameterized `mailbox #(T)`
Message types	Any, may be mixed	Exactly `T`
A wrong type is detected	At run time	At compile time
Blocking `get` / `peek` with a wrong type	Run-time error	Does not compile
`try_get` / `try_peek` with a wrong type	Returns a negative value	Does not compile
Readability	The reader cannot tell what travels through it	The type documents the connection
Recommended	No	Yes

> **Key point:** A `mailbox #(alu_txn)` can also carry objects of any class **derived** from `alu_txn`, because a derived-class handle may be stored in a base-class handle (Module 4). The receiver gets a base-class handle and uses `$cast` if it needs the derived type. This is how one driver can accept several kinds of transaction.

Interview questions

Q1. What is a parameterized mailbox and why is it preferred? A mailbox declared with a type parameter, `mailbox #(T)`, which accepts only messages of type `T`. Type errors are found at compile time. A typeless mailbox accepts anything and reports a mismatch only at run time, when a message is retrieved into a variable of the wrong type.

Q2. What happens if you `get()` from a typeless mailbox into a variable of the wrong type? A run-time error. With `try_get()` or `try_peek()` the function returns a negative value instead.

Q3. What are the three possible results of `try_get()`? Positive: a message was retrieved. Zero: the mailbox was empty. Negative: a message is available but its type does not match the argument (possible only with a typeless mailbox).

4.6 Mailbox of Transaction Objects

What is actually stored

When a class object is put into a mailbox, the mailbox stores the **handle**, not a copy of the object (Module 4: a class variable holds a reference). Sender and receiver end up with handles to the same object.

```
   generator:  txn = new();  txn.id = 1;  gen2drv_mbx.put(txn);

        txn (generator) ------+
                              v
                        +-----------+
                        | alu_txn   |
                        | id = 1    |
                        +-----------+
                              ^
        mailbox slot ---------+

   driver:  gen2drv_mbx.get(txn);      the driver's txn now refers to the SAME object
```

This is efficient, since no data is copied. It also causes the most common mailbox bug.

The bug: one object reused for every transaction

tb_mailbox_reuse_bug.sv

```
module tb_mailbox_reuse_bug;

  class alu_txn;
    bit [7:0]    a;
    int unsigned id;
  endclass

  mailbox #(alu_txn) gen2drv_mbx = new();
  alu_txn            txn, got;

  initial begin
    txn = new();                                // BUG: one object, created once

    for (int i = 1; i <= 3; i++) begin          // "generator"
      txn.id = i;
      txn.a  = 8'(i * 16);
      gen2drv_mbx.put(txn);                     // puts the same handle three times
    end

    repeat (3) begin                            // "driver"
      gen2drv_mbx.get(got);
      $display("driver got id=%0d a=%h", got.id, got.a);
    end
  end

endmodule
```

EXPECTED OUTPUT

```
driver got id=3 a=30
driver got id=3 a=30
driver got id=3 a=30
```

The generator intended to send three different transactions. The driver receives the last one three times.

```
   After the loop:

        mailbox slot 1 ---+
        mailbox slot 2 ---+----> +-----------+
        mailbox slot 3 ---+      | alu_txn   |      there is only ONE object;
        txn (generator) --+      | id = 3    |      each iteration overwrote its fields
                                 | a  = 30   |
                                 +-----------+
```

The fix: a new object for every transaction

```systemverilog
for (int i = 1; i <= 3; i++) begin
  txn    = new();                                  // FIX: construct inside the loop
  txn.id = i;
  txn.a  = 8'(i * 16);
  gen2drv_mbx.put(txn);
end
```

EXPECTED OUTPUT

```
driver got id=1 a=10
driver got id=2 a=20
driver got id=3 a=30
```

Each iteration now creates a separate object, and each mailbox slot refers to a different one.

An alternative with the same effect is to keep one "template" object, randomize it in each iteration, and put a **copy** into the mailbox: `gen2drv_mbx.put(txn.copy());`. This is the form used when a test replaces the template with a derived transaction type (Module 9).

> **Interview trap:** "My generator sends ten different transactions but the driver drives the same one ten times. Why?" The generator constructs the transaction once, outside the loop, and puts the same handle each time. With an unbounded mailbox the generator finishes the loop before the driver takes anything, so every handle in the mailbox points to the single object holding the last values. Construct a new object (or put a copy) for every transaction.

The same rule applies to the monitor: it must create a **new** `alu_txn` for each sampled transfer before putting it into `mon2scb_mbx`. Otherwise the scoreboard, which may still be holding the previous handle, sees its data change underneath it.

Interview questions

Q1. When a class object is put into a mailbox, is the object copied? No. Only the handle is stored. The receiver's handle refers to the same object the sender created.

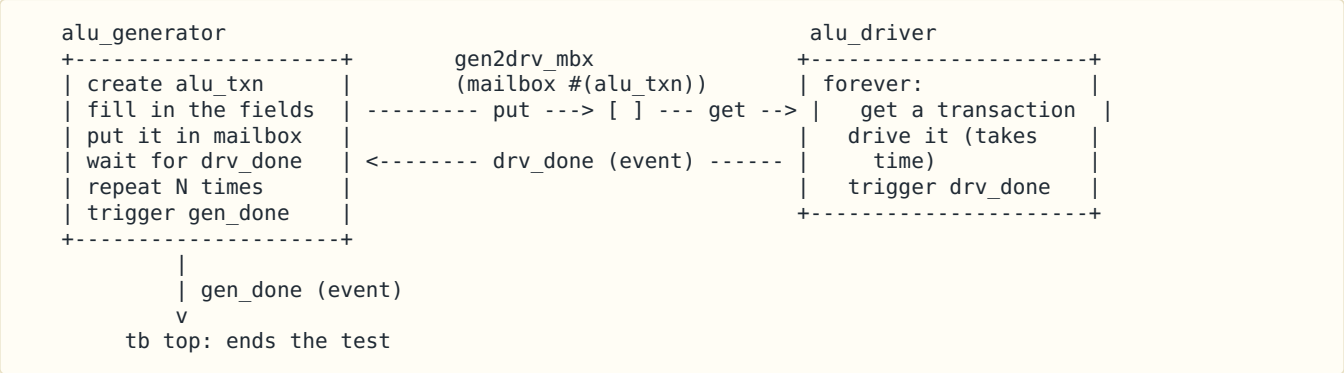
Q2. After `gen2drv_mbx.put(txn);` the generator modifies `txn.a`. Does the driver see the change? Yes, if the driver has not yet finished using the object, because both hold handles to the same object. To prevent this, the generator creates a new object for the next transaction instead of modifying the one it has sent.

Q3. Where should `new()` be called in a generator loop, and why? Inside the loop, once per transaction. If it is called once outside the loop, every `put` stores the same handle and all the messages refer to one object.

4.7 Communication Between Generator and Driver

This section combines a mailbox (for the data) and an event (for the handshake), using the component names of the course testbench. The classes are simplified: the generator sets field values by hand instead of randomizing (Module 5), and the driver consumes time with a delay instead of driving an interface (Module 8).

The design



- The **mailbox** carries the transactions from generator to driver.
- The event `drv_done` goes the other way: the driver tells the generator that it has finished the current transaction. The generator therefore creates one transaction at a time, in step with the driver.
- The event `gen_done` tells the top level that the generator has sent everything.

Complete example

tb_gen_drv_mailbox.sv

```
module tb_gen_drv_mailbox;

  typedef enum logic [2:0] {
    ALU_ADD, ALU_SUB, ALU_AND, ALU_OR, ALU_XOR, ALU_NOT, ALU_SHL, ALU_SHR
  } alu_op_e;

  // ---------------------------------------------------------------- transaction
  class alu_txn;
    bit [7:0]    a, b;
    alu_op_e     op;
    int unsigned id;

    function void print(string tag = "");
      $display("[%0t] %s id=%0d a=%0d b=%0d op=%s", $time, tag, id, a, b, op.name());
    endfunction
  endclass

  // ---------------------------------------------------------------- generator
  class alu_generator;
    mailbox #(alu_txn) gen2drv_mbx;
    event              drv_done;
    event              gen_done;
    int unsigned       num_txn = 3;

    function new(mailbox #(alu_txn) gen2drv_mbx, event drv_done);
      this.gen2drv_mbx = gen2drv_mbx;
      this.drv_done    = drv_done;
    endfunction

    task run();
      alu_txn txn;
      for (int unsigned i = 1; i <= num_txn; i++) begin
        txn    = new();                       // a new object for every transaction
```

```
        txn.id = i;
        txn.a  = 8'(10 * i);
        txn.b  = 8'(i);
        txn.op = (i % 2) ? ALU_ADD : ALU_SUB;
        txn.print("GEN: put");
        gen2drv_mbx.put(txn);
        @(drv_done);                          // wait until the driver has finished it
      end
      $display("[%0t] GEN: all %0d transactions sent", $time, num_txn);
      -> gen_done;
    endtask
  endclass

  // ---------------------------------------------------------------------- driver
  class alu_driver;
    mailbox #(alu_txn) gen2drv_mbx;
    event              drv_done;

    function new(mailbox #(alu_txn) gen2drv_mbx, event drv_done);
      this.gen2drv_mbx = gen2drv_mbx;
      this.drv_done    = drv_done;
    endfunction

    task run();
      alu_txn txn;
      forever begin
        gen2drv_mbx.get(txn);                 // blocks until a transaction arrives
        txn.print("DRV: got");
        #10;                                  // stands for driving the DUT pins
        $display("[%0t] DRV: done id=%0d", $time, txn.id);
        -> drv_done;
      end
    endtask
  endclass

  // ---------------------------------------------------------------------- top level
  mailbox #(alu_txn) gen2drv_mbx;
  event              drv_done;
  alu_generator      gen;
  alu_driver         drv;

  initial begin
    gen2drv_mbx = new();                      // ONE mailbox ...
    gen = new(gen2drv_mbx, drv_done);         // ... shared by both components
    drv = new(gen2drv_mbx, drv_done);

    fork
      gen.run();
      drv.run();
    join_none

    @(gen.gen_done);
    $display("[%0t] TOP: generator finished, %0d items left in the mailbox",
             $time, gen2drv_mbx.num());
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[0] GEN: put id=1 a=10 b=1 op=ALU_ADD
[0] DRV: got id=1 a=10 b=1 op=ALU_ADD
[10] DRV: done id=1
[10] GEN: put id=2 a=20 b=2 op=ALU_SUB
[10] DRV: got id=2 a=20 b=2 op=ALU_SUB
[20] DRV: done id=2
[20] GEN: put id=3 a=30 b=3 op=ALU_ADD
[20] DRV: got id=3 a=30 b=3 op=ALU_ADD
[30] DRV: done id=3
[30] GEN: all 3 transactions sent
[30] TOP: generator finished, 0 items left in the mailbox
```

Execution flow

Time	Generator	Driver	Top
0	Creates txn 1, prints, `put`. Blocks at `@(drv_done)`	`get` returns txn 1. Prints. Starts "driving" (`#10`)	Forked both, now blocked at `@(gen.gen_done)`
10		Prints "done", triggers `drv_done`, loops to `get`: mailbox empty, blocks	
10	Wakes. Creates txn 2, prints, `put`. Blocks at `@(drv_done)`	`get` returns txn 2. Prints. Drives until 20	
20	Same for txn 3	Same for txn 3	
30	Wakes, loop finished. Prints. Triggers `gen_done`, ends	Blocked in `get`	Wakes, prints, `$finish`

Every line of the output follows from the previous one by cause and effect, so the order does not depend on the simulator: the driver cannot print "got" before the generator's `put`, and the generator cannot print the next "put" before the driver's `-> drv_done`.

Line by line

Code	Explanation
`mailbox #(alu_txn) gen2drv_mbx;` in both classes	Each class has a **handle**. Neither class constructs a mailbox.
`function new(mailbox #(alu_txn) gen2drv_mbx, event drv_done);`	The handles are passed in when the object is built, so a component cannot exist unconnected.
`this.gen2drv_mbx = gen2drv_mbx;`	`this.` selects the class property; the plain name is the constructor argument (Module 4).
`txn = new();` inside the loop	Avoids the reuse bug of Section 4.6.
`txn.op = (i % 2) ? ALU_ADD : ALU_SUB;`	Odd identifiers get `ALU_ADD`, even ones `ALU_SUB`. Both branches are of the enum type, so no cast is needed.
`gen2drv_mbx.put(txn);`	The mailbox is unbounded, so this never blocks.
`@(drv_done);` in the generator	The generator sleeps until the driver has finished. This `@` is safe from the race of Section 2.2 because the driver triggers only after a `#10`: the generator is always waiting long before the trigger.
`forever begin gen2drv_mbx.get(txn); ...`	The standard driver loop. `get` is the only thing that paces it; when no transaction is available the driver sleeps.
`-> drv_done;`	Tells the generator that the item is complete.
`gen2drv_mbx = new();` at the top	The single mailbox object. Both components receive the same handle.
`fork gen.run(); drv.run(); join_none`	Two threads. `join` could not be used: the driver's `forever` loop never ends.
`@(gen.gen_done);`	The top level reaches the generator's event through the generator handle. It starts waiting at time 0, well before the trigger at time 30.
`$finish;`	Ends the simulation; the driver, blocked in `get`, is simply abandoned.

Other ways to keep generator and driver in step

Technique	How	Comment
Event handshake (above)	Generator waits for `drv_done` after each `put`	Explicit and easy to follow
Bounded mailbox of size 1	`gen2drv_mbx = new(1);`	The generator can still be one transaction ahead: its `put` succeeds as soon as the driver has done `get`, before the driver has finished driving
Bounded mailbox of size 1 with `peek`	Driver does `peek`, drives, then `get`	The message stays in the mailbox while it is being driven, so the generator's next `put` blocks until the driver has completely finished. True lock-step without an event
No synchronization	Unbounded mailbox, generator puts everything at once	Simplest; acceptable when the stimulus does not depend on what the DUT is doing

```
// Driver loop for the peek-then-get technique (mailbox created with new(1))
forever begin
  gen2drv_mbx.peek(txn);      // look at the transaction; the mailbox stays full
  drive(txn);                 // takes simulation time
  gen2drv_mbx.get(txn);       // remove it: only now can the generator put the next one
end
```

> **Verification application:** This generator → mailbox → driver structure is half of the layered testbench built in Module 9. The other half, monitor → `mon2scb_mbx` → scoreboard, is identical in form. In UVM the mailbox and the handshake are replaced by the sequencer-driver connection (`get_next_item()` and `item_done()`), which works on the same principle.

Interview questions

Q1. How do a generator and a driver communicate? Through a mailbox of transaction handles. The environment constructs one mailbox and passes its handle to both. The generator creates (and randomizes) a transaction and calls `put()`; the driver calls `get()` in a `forever` loop and drives each transaction onto the interface.

Q2. How does the generator know when the driver has finished a transaction? Through a handshake: an event such as `drv_done` that the driver triggers after driving and the generator waits for after each `put`. Alternatives are a bounded mailbox of size 1 combined with `peek` then `get` in the driver.

Q3. Who should construct the mailbox, and why? The environment (or whatever owns both components). There must be exactly one mailbox object, with its handle given to the generator and the driver. If each component constructs its own, they are not connected.

Q4. Why is `fork` ... `join_none` (or `join_any`) used to start the generator and driver, and not `join`? The driver runs a `forever` loop and never returns, so `join` would never complete.

Q5. With a bounded mailbox of size 1 and a driver that uses `get` first and then drives, is the generator in lock-step with the driver? Not quite. As soon as the driver has done `get`, the mailbox is empty and the generator's next `put` succeeds, so the generator is one transaction ahead. For true lock-step the driver uses `peek`, drives, and then `get`, or an event handshake is added.

4.8 Mailbox: Summary, Mistakes and Questions

Common mistakes

Mistake	Symptom	Fix
Mailbox never constructed	Null-handle run-time error at the first `put` / `get`	`mbx = new();`
Two mailboxes constructed, one in each component	Generator sends, driver waits forever	Construct once, share the handle
Same object put repeatedly	Receiver sees the last value every time	`new()` per transaction, or put a copy
Typeless mailbox	Run-time type error	`mailbox #(T)`
`try_get` in a loop with no delay	Simulation hangs at one time	Blocking `get`, or add a clock edge
Ignoring the return value of `try_put` / `try_get`	Messages silently not sent, or stale data processed	Test the return value
`get` called from a function	Compile error	`try_get`, or make it a task
Bounded mailbox where the consumer has stopped	Producer blocks forever (deadlock)	End-of-test logic, timeouts
Two consumers on one mailbox, each expecting to see every message	Each message is delivered to only one of them	One mailbox per consumer; the producer puts into each

Further interview questions

Q1. Can several threads `get` from the same mailbox? Yes, but each message is delivered to exactly one of them. A mailbox does not broadcast. To give every consumer every message, use a separate mailbox for each consumer and have the producer put a handle into each.

Q2. Can a mailbox be passed to a task or stored in an array? Yes. It is a class handle, so it can be an argument, a class property, an array element, or the value of an associative array.

Q3. Is the mailbox FIFO guaranteed? Yes. Messages are retrieved in the order in which they were put.

Q4. How do you empty a mailbox at the end of a test, from a function? `while (mbx.try_get(item)) begin ... end`. `try_get` is a function, never blocks, and returns 0 when the mailbox is empty.

Q5. What is the difference between a mailbox and a semaphore? A mailbox transfers data: it is a queue of messages. A semaphore transfers no data: it holds a count of keys and is used to control access to a resource. Both block a thread when the request cannot be satisfied (Part 5).

Q6. How would you implement a mailbox yourself? With a queue for the messages, a `put` task that does `wait (q.size() < bound);` (when bounded) and then `q.push_back(msg);`, and a `get` task that does `wait (q.size() > 0);` and then `msg = q.pop_front();`. The built-in mailbox additionally preserves the arrival order of blocked threads and is safe when several threads are woken at the same time.

Part 5: Semaphore

5.1 What a Semaphore Is and Why It Is Required

Start with the problem

Two drivers in a testbench are connected to the same bus. Each one, when it has a transfer to send, puts its data on the bus and holds it there for a while. Nothing stops both from doing so at the same time.

tb_bus_no_semaphore.sv

```
module tb_bus_no_semaphore;

  logic [7:0] bus;

  task automatic send(string name, logic [7:0] data, int unsigned hold);
    bus = data;
    $display("[%0t] %s drives %h", $time, name, data);
    #hold;
    $display("[%0t] %s finished: bus = %h, expected %h", $time, name, bus, data);
  endtask

  initial begin
    fork
      send("DRV_A", 8'hAA, 20);
      begin
        #5;
        send("DRV_B", 8'hBB, 20);
      end
    join
  end

endmodule
```

EXPECTED OUTPUT

```
[0] DRV_A drives aa
[5] DRV_B drives bb
[20] DRV_A finished: bus = bb, expected aa
[25] DRV_B finished: bus = bb, expected bb
```

Driver A's transfer was destroyed at time 5, when driver B overwrote the bus in the middle of it. Neither driver knows about the other. The bus is a **shared resource**, and access to it must be **mutually exclusive**: one user at a time. The piece of code that uses the resource (from "start driving" to "finished") is called a **critical section**.

Other shared resources in a testbench:

- one physical interface used by several sequences or agents;
- a register bus used both by the test and by an interrupt-handling thread;
- a shared memory model or a shared counter updated by several threads;
- a log file written by several threads;
- a limited pool of something (for example at most two outstanding requests).

In plain English

A semaphore is a box of keys beside a locked room. To enter the room you must take a key from the box. If the box is empty you wait beside it. When you leave the room you put the key back, and the next person waiting takes it. With one key in the box, only one person can be in the room at a time. With three keys, up to three.

Technical definition

A **semaphore** is a built-in SystemVerilog class that holds a count of **keys**.

- `get(n)` takes `n` keys. If fewer than `n` are available, the calling thread **blocks** until they are.
- `put(n)` returns `n` keys to the semaphore and may unblock waiting threads.
- A thread holding the key(s) has the right to use the resource the semaphore protects.

A semaphore created with one key is a **mutex** (mutual exclusion lock, also called a binary semaphore). A semaphore with several keys is a **counting semaphore**.

A semaphore is a class, so a semaphore variable is a handle that is `null` until `new()` is called, and several components share a semaphore by sharing its handle.

How it works internally

```
            semaphore object
          +---------------------------+
          |  key count : 1            |   get(1): if count >= 1 -> count - 1, continue
          |  waiters   : (FIFO list)  |           else           -> wait in the queue
          +---------------------------+   put(1): count = count + 1, wake waiters that can
                                                  now be satisfied

time   DRV_A                key count        DRV_B
----   -----                ---------        -----
 0     get()  -> granted       1 -> 0
 5                                           get()  -> no key: blocked
 20    put()                   0 -> 1
                               1 -> 0        (woken) granted
 40                            0 -> 1        put()
```

The important design fact: **a semaphore contains only a number**. It does not know which thread holds a key, and the keys have no identity. This has consequences that are covered in Section 5.4.

Interview questions

Q1. What is a semaphore? A built-in class that holds a number of keys and is used to control access to a shared resource. A thread must obtain a key with `get()` before using the resource and return it with `put()` afterwards. If no key is available, `get()` blocks.

Q2. Why is a semaphore required? When several threads use the same resource (a bus, an interface, a shared variable), their operations can overlap and corrupt each other. A semaphore makes access mutually exclusive, or limits it to a fixed number of simultaneous users.

Q3. What is a critical section? The portion of code that accesses the shared resource and must therefore be executed by only one thread at a time. It lies between the `get()` and the `put()`.

Q4. What is the difference between a binary and a counting semaphore? A binary semaphore (mutex) has one key, so only one thread can hold it. A counting semaphore has several keys and allows that many threads, or one thread taking several keys, to proceed at once.

5.2 Declaration and Methods: new, get, put, try_get

Syntax

```
semaphore sem;              // a handle, null until constructed

sem = new(1);               // create with 1 key   (a mutex)
sem = new(3);               // create with 3 keys
sem = new();                // create with 0 keys  (the default)

sem.get();                  // take 1 key; block if none is available
sem.get(2);                 // take 2 keys; block until 2 are available
sem.put();                  // return 1 key
sem.put(2);                 // return 2 keys

if (sem.try_get())  ...     // take 1 key if available; never blocks
if (sem.try_get(2)) ...     // take 2 keys if 2 are available; never blocks
```

The methods

Method	Kind	Blocks	What it does	Return value
`new(int keyCount = 0)`	Constructor	No	Creates the semaphore with `keyCount` keys	The handle
`get(int keyCount = 1)`	Task	Yes, until that many keys are available	Removes `keyCount` keys	None
`put(int keyCount = 1)`	Function	No	Adds `keyCount` keys; waiting threads may be released	None
`try_get(int keyCount = 1)`	Function	No	Removes `keyCount` keys if that many are available	Positive if the keys were taken, 0 otherwise

Points to note:

- The default number of keys in `new()` is **0**. A semaphore created with `new()` and no argument blocks the first `get()`. For a mutex write `new(1)`.
- `put()` is a function: it never blocks and can be called from a function. (A mailbox's `put()` is a task, because it can block.)
- There is **no `try_put`**, because `put` cannot fail, and **no method to read the number of keys**.
- `get(n)` is all-or-nothing: it never takes some of the keys and waits for the rest.

Complete example

tb_semaphore_keys.sv

```
module tb_semaphore_keys;

  semaphore sem;

  initial begin
    sem = new(3);                                          // 3 keys

    if (sem.try_get(2))  $display("1. got 2 keys              : 1 left");
    if (!sem.try_get(2)) $display("2. cannot get 2 keys       : only 1 left, none taken");
    if (sem.try_get(1))  $display("3. got the last key        : 0 left");
    if (!sem.try_get())  $display("4. cannot get a key        : 0 left");

    sem.put(3);
    $display("5. returned 3 keys         : 3 available");

    sem.put(2);
    $display("6. put 2 keys never taken  : 5 available (created with 3)");

    if (sem.try_get(5))  $display("7. got 5 keys at once      : 0 left");
  end

endmodule
```

EXPECTED OUTPUT

```
1. got 2 keys              : 1 left
2. cannot get 2 keys       : only 1 left, none taken
3. got the last key        : 0 left
4. cannot get a key        : 0 left
5. returned 3 keys         : 3 available
6. put 2 keys never taken  : 5 available (created with 3)
7. got 5 keys at once      : 0 left
```

Line by line

Line	Explanation
`sem = new(3);`	The semaphore starts with three keys.
1	`try_get(2)` finds three keys, takes two, returns a positive value.
2	Only one key remains. `try_get(2)` takes nothing and returns 0. A blocking `get(2)` here would suspend the thread forever, because no other thread exists to return keys.
3, 4	The last key is taken; a further request fails. `try_get()` with no argument asks for one key.
5	Three keys are returned.
6	Two **more** keys are put, although this thread holds none. The semaphore accepts them: it now has five keys, more than it was created with.
7	Proof that five keys exist: a request for five succeeds.

`try_get` is used throughout this example because the example has a single thread: a blocking `get` that could not be satisfied would never be released.

Key-based synchronization: the standard pattern

```
sem.get(1);          // acquire the key        : entering the critical section
// ... use the shared resource; this may take simulation time ...
sem.put(1);          // release the key        : leaving the critical section
```

Every thread that uses the resource must follow this pattern, with the same semaphore. The protection is a convention: nothing in the language stops a thread from using the resource without taking the key.

Interview questions

Q1. What are the semaphore methods? `new()`, `get()`, `put()` and `try_get()`.

Q2. What is the default number of keys? Zero for `new()` without an argument. The default number of keys requested or returned by `get()`, `put()` and `try_get()` is one.

Q3. What is the difference between `get()` and `try_get()`? `get()` is a blocking task: it waits until the requested keys are available. `try_get()` is a function that never blocks: it takes the keys and returns a positive value if they are available, and returns 0 otherwise.

Q4. Can `put()` block? No. It is a function and always completes immediately.

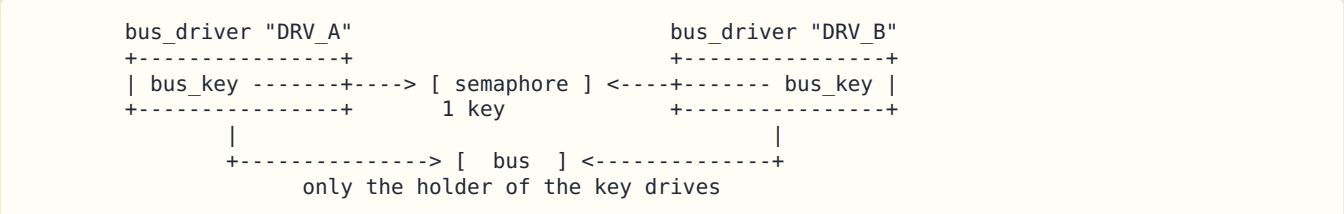
Q5. A semaphore has one key and a thread calls `get(2)`. What happens? The thread blocks until two keys are available. It does not take the single key in the meantime. If nobody ever puts another key, it blocks forever.

Q6. Is there a way to ask a semaphore how many keys it has? No. The only way to test availability is `try_get()`, which also takes the keys if they are there.

5.3 Resource Sharing: Two Drivers on One Bus

The design

The two drivers of Section 5.1 now share a semaphore with one key. A driver must hold the key for the whole time it is using the bus.



Complete example

`tb_semaphore_bus.sv`

```
module tb_semaphore_bus;

  logic [7:0] bus;

  class bus_driver;
    string    name;
    semaphore bus_key;                          // handle to the shared semaphore

    function new(string name, semaphore bus_key);
      this.name    = name;
```

```
      this.bus_key = bus_key;
    endfunction

    task send(logic [7:0] data, int unsigned hold);
      $display("[%0t] %s requests the bus", $time, name);
      bus_key.get(1);                          // blocks while the other driver has it
      $display("[%0t] %s granted", $time, name);
      bus = data;                              // ---- critical section starts ----
      #hold;
      $display("[%0t] %s releasing, bus = %h", $time, name, bus);
      bus_key.put(1);                          // ---- critical section ends ------
    endtask
  endclass

  semaphore  bus_key;
  bus_driver drv_a, drv_b;

  initial begin
    bus_key = new(1);                          // ONE semaphore with ONE key
    drv_a   = new("DRV_A", bus_key);
    drv_b   = new("DRV_B", bus_key);

    fork
      begin
        drv_a.send(8'hAA, 20);
        #3;
        drv_a.send(8'hAA, 20);
      end
      begin
        #5;
        drv_b.send(8'hBB, 20);
        #7;
        drv_b.send(8'hBB, 20);
      end
    join
    $display("[%0t] TOP: all transfers complete", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] DRV_A requests the bus
[0] DRV_A granted
[5] DRV_B requests the bus
[20] DRV_A releasing, bus = aa
[20] DRV_B granted
[23] DRV_A requests the bus
[40] DRV_B releasing, bus = bb
[40] DRV_A granted
[47] DRV_B requests the bus
[60] DRV_A releasing, bus = aa
[60] DRV_B granted
[80] DRV_B releasing, bus = bb
[80] TOP: all transfers complete
```

Execution flow

Time	DRV_A	DRV_B	Keys	Bus owner
0	Requests, key available: granted		1 → 0	A
5	Driving	Requests, no key: **blocked**	0	A
20	Prints "releasing", `put`	Woken by the `put` : granted	0 → 1 → 0	B
23	Requests, no key: **blocked**	Driving	0	B
40	Woken: granted	Prints "releasing", `put`	0 → 1 → 0	A

Time	DRV_A	DRV_B	Keys	Bus owner
47	Driving	Requests, no key: **blocked**	0	A
60	Prints "releasing", `put`	Woken: granted	0 → 1 → 0	B
80		Prints "releasing", `put`	0 → 1	none

Each "releasing" line shows the bus still holding the value its owner wrote: no transfer was disturbed. Compare with Section 5.1, where driver A found `bb` on the bus.

Line by line

Code	Explanation
`semaphore bus_key;` in the class	A handle. The class does not create a semaphore.
`function new(string name, semaphore bus_key);`	The shared semaphore is passed in at construction.
`bus_key = new(1);` at the top	The one semaphore, with one key. If each driver constructed its own, each would have its own key and there would be no exclusion.
`bus_key.get(1);`	Blocks until the key is available. At time 5 this is where DRV_B stops.
`bus = data; #hold;`	The critical section. It consumes time, which is exactly why protection is needed.
`$display(... "releasing" ...)` before `bus_key.put(1);`	The message is printed before the key is returned, so it always appears before the other driver's "granted" message.
`bus_key.put(1);`	Returns the key. The waiting driver becomes ready and runs when the current thread blocks or ends.
`fork ... join`	Each driver's sequence of two transfers is one thread. `join` waits for both.

> **Verification application:** In a real testbench the `get` / `put` pair is placed inside the driver's or the bus-functional model's transfer task, so that every user of the interface is protected automatically: `task write(...); bus_key.get(); ...drive...; bus_key.put(); endtask`. UVM offers the same service for sequences through `lock()` / `unlock()` and `grab()` / `ungrab()` on a sequencer.

Interview questions

Q1. Two drivers share one interface. How do you prevent them from driving at the same time? Create one semaphore with one key and give both drivers its handle. Each driver calls `get()` before driving and `put()` when its transfer is complete. While one holds the key the other blocks in `get()`.

Q2. Where should the semaphore be constructed? Once, in the component that owns both users (the environment or agent), and its handle passed to each user. Constructing one per driver gives each its own key and provides no protection.

Q3. In what order are threads that are waiting for a key served? In the order in which they started waiting: the semaphore's waiting queue is first-in, first-out.

5.4 Semaphore Pitfalls

1. Keys can be put that were never taken

`put()` simply adds to the count. Example 5.2 ended with five keys in a semaphore created with three. For a mutex this is serious:

```
bus_key = new(1);
...
bus_key.put();        // a stray put (for example an error path "releasing" without a get)
// The semaphore now has 2 keys. Two drivers can hold the "mutex" at the same time.
```

No error or warning is given. The symptom is intermittent corruption of the shared resource long after the stray `put`.

2. A key that is never returned

If a thread takes the key and leaves the critical section without `put()`, every other user blocks forever.

```
task send(alu_txn txn);
  bus_key.get();
  if (txn == null) return;          // BUG: returns while holding the key
  drive(txn);
  bus_key.put();
endtask
```

The same happens if the thread holding the key is killed by `disable fork` in the middle of the critical section. Every path out of the critical section must return the key.

3. No ownership

The semaphore does not record who holds a key. Any thread can call `put()`, including one that never called `get()`. A thread that calls `get()` twice on a mutex blocks on itself forever (the semaphore is not "re-entrant").

4. Asking for more keys than can ever exist

`sem = new(2); sem.get(3);` blocks forever unless some thread puts an extra key.

5. Several keys and waiting order

When threads request different numbers of keys, the order in which they are served is delicate. Suppose thread X is waiting for two keys, thread Y arrives later and waits for one key, and then one key is put. A strict first-in, first-out queue keeps Y waiting behind X; an implementation that serves any request it can satisfy lets Y through. The standard states that the waiting queue is FIFO, but simulators have differed in this corner. Do not write code whose correctness depends on it. If all users request one key, the question does not arise.

6. A semaphore with zero keys used as a signal

```
semaphore done = new(0);      // no keys

// notifier                         // waiter
done.put();                         done.get();     // passes if the put has happened
                                                    // at ANY earlier time
```

Unlike an event, the key is **remembered**: the waiter does not miss the notification if it arrives late, and several notifications are counted. This is a legitimate use when a persistent "it has happened" signal is needed.

Summary of pitfalls

Pitfall	Symptom	Prevention
`new()` instead of `new(1)`	First `get()` blocks forever	Always give the key count
Extra `put()`	Mutual exclusion silently lost	Exactly one `put` for every `get`
Missing `put()` on some path	Other threads hang	Release on every exit path
`get()` twice by the same thread	That thread deadlocks	Do not nest acquisitions of the same mutex
Thread killed while holding a key	Other threads hang	Do not `disable` threads inside critical sections
One semaphore per user	No protection	One shared semaphore
Polling with `try_get()` in a zero-time loop	Simulation hangs	Blocking `get()`, or a delay in the loop

Interview questions

Q1. A semaphore is created with one key. A thread calls `put()` without calling `get()`. What happens? The semaphore now has two keys. No error is reported. Two threads can then acquire the "mutex" simultaneously, so mutual exclusion is broken.

Q2. Does a semaphore know which process holds its key? No. It only keeps a count. Any process may put keys, whether or not it took any.

Q3. What happens if a thread forgets to call `put()`? The key is never returned, and every thread that subsequently calls `get()` blocks forever: a deadlock.

Q4. How can a semaphore be used instead of an event? Create it with zero keys. The notifying thread calls `put()`, and the waiting thread calls `get()`. Because the key persists, the waiter is released even if it starts waiting after the notification, which an event cannot do.

5.5 Semaphore versus Mailbox, and Summary

Difference between semaphore and mailbox

Property	Semaphore	Mailbox
Purpose	Control access to a shared resource	Transfer data between threads
What it holds	A count of identical keys	A FIFO queue of messages
Carries data	No	Yes
`get`	Takes keys; blocks if not enough	Takes the oldest message; blocks if empty
`put`	Returns keys; **never blocks** (function)	Adds a message; blocks if a bounded mailbox is full (task)

Property	Semaphore	Mailbox
`peek`	Not available	Available
Nonblocking methods	`try_get` only	`try_get`, `try_put`, `try_peek`
Query	None	`num()`
Upper limit	None: keys can always be added	Optional bound
Typical relationship between users	The same thread does `get` then `put`	One thread does `put`, another does `get`
Type parameter	None	`mailbox #(T)`
Typical use	Two sequences sharing one driver or interface	Generator → driver, monitor → scoreboard

Both are built-in classes, both must be constructed with `new()`, both are shared by sharing the handle, and both keep waiting threads in arrival order.

> **Key point:** A semaphore behaves like a mailbox whose messages have no content. Indeed, a mutex can be imitated with a bounded mailbox of size 1 (`put` to lock, `get` to unlock), but a semaphore states the intention directly.

Common mistakes

Mistake	Fix
Using a semaphore to pass data	Use a mailbox
Using a mailbox or an event to protect a resource	Use a semaphore
Holding the key longer than necessary (for example across a wait for a response that needs the other user)	Keep the critical section minimal
Acquiring two semaphores in different orders in different threads	Always acquire in the same order (Section 6.5)

Further interview questions

Q1. What is the difference between a semaphore and a mailbox? A semaphore holds a count of keys and is used for mutual exclusion or for limiting the number of simultaneous users; it carries no data, and its `put` never blocks. A mailbox is a FIFO of messages used to pass data from one thread to another; its `get` blocks when empty and its `put` blocks when a bounded mailbox is full.

Q2. How would you allow at most two of five threads to be active at once? Create a semaphore with two keys. Each thread calls `get()` before the activity and `put()` afterwards. At most two can hold a key at any time.

Q3. Can a semaphore be used inside a function? `put()` and `try_get()` can, because they are functions. `get()` is a task and cannot be called from a function.

Q4. A thread needs two resources, each protected by its own semaphore. What must you be careful about? Deadlock. If one thread takes semaphore A then B, and another takes B then A, each can end up holding one and waiting for the other. All threads must acquire the semaphores in the same order.

Q5. Is a SystemVerilog semaphore a mutex? With one key and disciplined use it serves as a mutex, but it is weaker than a true mutex: it has no owner, so it cannot detect a release by a thread that does not hold the key, and it can be given extra keys.

Part 6: Timing and Synchronization

6.1 #delay versus @event versus wait

In plain English

A thread can be told to pause in three different ways.

- `#10` : "sleep for 10 time units." The thread sets an alarm clock.
- `@(something)` : "sleep until `something` **changes**." The thread waits for a doorbell.
- `wait (condition)` : "if the condition is already true, carry on; otherwise sleep until it **is** true." The thread checks whether the door is open, and waits only if it is closed.

Technical definition

Timing control	Name	Resumes when
`#delay`	Delay control	The given amount of simulation time has elapsed
`@(event_expression)`	Event control (edge-sensitive)	The expression **changes** after the thread has started waiting: an edge of a signal, any change of a value, or the trigger of a named event
`wait (expression)`	Wait statement (level-sensitive)	The expression is **true**. If it is already true the thread does not suspend at all

Forms of each

```
// ---- delay control ----
#10;                          // 10 time units
#(2 * period);                // an expression
#1ns;                         // a time literal with a unit
#0;                           // zero delay: reschedule in the Inactive region (avoid)

// ---- event control ----
@(posedge clk);               // rising edge
@(negedge rst_n);             // falling edge
@(data);                      // any change of data
@(a or b);   @(a, b);         // a change of a or of b
@(posedge clk iff valid);     // a rising edge at which valid is true
@(ev);                        // a named event
repeat (3) @(posedge clk);    // three rising edges
@(vif.mon_cb);                // a clocking-block event (Module 8)

// ---- wait ----
wait (ready);                 // until ready is non-zero
wait (count == 8);            // until an expression is true
wait (q.size() > 0);          // until a queue is not empty
wait (ev.triggered);          // until an event has triggered in this time step
wait fork;                    // until all child threads have finished (Section 3.7)
wait_order (a, b, c);         // until events have occurred in order (Section 2.6)
```

Complete example

tb_timing_controls.sv

```
module tb_timing_controls;

  logic ready = 0;

  initial begin : stimulus
    #10 ready = 1;            // ready is 1 from time 10 ...
    #10 ready = 0;            // ... 0 from time 20 ...
    #10 ready = 1;            // ... and 1 again from time 30
  end

  initial begin : level_wait
    #15;
    wait (ready);
    $display("[%0t] wait (ready) passed", $time);
  end

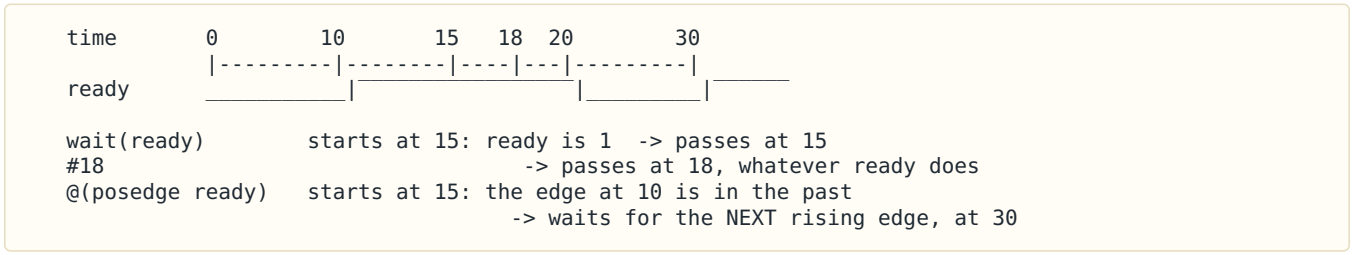
  initial begin : edge_wait
    #15;
    @(posedge ready);
    $display("[%0t] @(posedge ready) passed", $time);
  end

  initial begin : delay_wait
    #18;
    $display("[%0t] #18 passed", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[15] wait (ready) passed
[18] #18 passed
[30] @(posedge ready) passed
```



Line by line

Thread	Explanation
`level_wait`	At time 15 `ready` is already 1. `wait` is level-sensitive, so it does not block.
`delay_wait`	A delay looks at nothing. It resumes at time 18.
`edge_wait`	At time 15 `ready` is 1, but `@(posedge ready)` needs a **transition** that happens after it starts waiting. The rising edge at 10 is over, so it waits for the one at 30.

Comparison table

Property	`#delay`	`@(event)`	`wait (expr)`
Waits for	Time to pass	A change	A condition to be true

Property	`#delay`	`@(event)`	`wait (expr)`
Sensitivity	None	Edge	Level
If the awaited thing has already happened	Not applicable	**Missed**; waits for the next occurrence	**Seen**; continues immediately
Can complete in zero time	Only `#0`	No: always suspends at least until the next change	Yes
Depends on design activity	No	Yes	Yes
Risk	Fragile if the DUT's timing changes	Missing an event that occurred earlier	An endless zero-time loop if used in a loop while the condition stays true
Typical use	Clock generation, reset width, timeouts, modelling delay	Synchronising to clock edges; repeated notifications	Waiting for a status or flag; one-shot notifications
Synthesizable	No	Only as the sensitivity list of an `always` block	No

Interview trap: `forever begin wait (clk); count++; end` is not a clock-edge counter. Once `clk` is 1 the `wait` passes immediately in every iteration, time never advances, and the simulation hangs. To count edges, use `@(posedge clk)`.

Common mistake: Using delays to synchronise with the DUT: `#50; check_result();`. If the DUT's latency changes by one cycle the check samples the wrong value. Synchronise to what you are actually waiting for: `@(posedge clk iff valid_out);` or `wait (vif.valid_out);`.

Key point: `@(posedge sig)` detects any transition toward 1: 0→1, but also 0→X, 0→Z, X→1 and Z→1. The first clock edge after an X at time 0 can therefore be an unexpected "posedge".

Interview questions

Q1. What is the difference between `#`, `@` and `wait`? `#` suspends the process for a fixed amount of simulation time. `@` suspends it until a specified change occurs, and is edge-sensitive: the change must happen after the process started waiting. `wait` suspends it until an expression is true, and is level-sensitive: if the expression is already true, the process continues without suspending.

Q2. `a` is already 1. What do `@(posedge a)` and `wait (a)` do? `@(posedge a)` blocks until `a` goes low and then high again. `wait (a)` continues immediately.

Q3. What does `@(posedge clk iff valid)` mean? Wait for a rising edge of `clk` at which `valid` is true. Edges at which `valid` is false are ignored.

Q4. Which of the three can be synthesised? None as a procedural statement. An event control is synthesizable only as the sensitivity of an `always` block (`always @(posedge clk)`). Delays are ignored by synthesis and `wait` is not supported.

6.2 @event versus wait(event.triggered), and -> versus ->>

These two comparisons were developed in Sections 2.2 and 2.3. Here they are combined, because the choices on the triggering side and on the waiting side interact.

The situation

A trigger and a wait occur **in the same time step**, in different threads, and the trigger is executed first.

Trigger used	Wait used	Does the waiter continue in this time step?	Why
`-> ev`	`@(ev)`	**No** (race lost; it would have continued if it had run first)	The trigger was over before the wait began
`-> ev`	`wait (ev.triggered)`	Yes	`triggered` stays true to the end of the time step
`->> ev`	`@(ev)`	Yes	The trigger is postponed to the NBA region; the waiter is waiting by then
`->> ev`	`wait (ev.triggered)`	Yes	Either mechanism is sufficient

Only the first row is unsafe, and it is the one people write by default.

Side-by-side summary

	`@(ev)`	`wait (ev.triggered)`
Kind	Edge-sensitive event control	Level-sensitive wait on a property
Catches a trigger made earlier in the same time step	No	Yes
Catches a trigger made in an earlier time step	No	No
Safe in a loop without a time delay	Yes	No

	`-> ev`	`->> ev`
Kind	Immediate (blocking) trigger	Nonblocking trigger
Takes effect	Now, in the Active region	In the NBA region, or after the stated delay
Suspends the triggering thread	No	No
Accepts a delay: `->> #5 ev`	No	Yes

Choosing

Situation	Recommended
One-shot notification ("reset done", "generator done")	`-> ev` with `wait (ev.triggered)`
Repeated notification consumed in a loop	`@(ev)`, with a protocol that guarantees the waiter is waiting first; or `->> ev`
The notification must not be lost even if the waiter arrives much later	Not an event: a flag, a semaphore with zero keys, or a mailbox

Interview questions

Q1. Give two ways to avoid losing an event when the trigger and the wait happen at the same simulation time. Use `wait (ev.triggered)` instead of `@(ev)` on the waiting side, or use the nonblocking trigger `->> ev` instead of `-> ev` on the triggering side.

Q2. Which combination of trigger and wait is exposed to a race? `-> ev` together with `@(ev)`.

6.3 Blocking versus Nonblocking Synchronization

Definition

- A **blocking** operation may suspend the calling thread until it can be completed. The thread makes no progress in the meantime. Blocking operations are **tasks** or timing statements.
- A **nonblocking** operation always returns immediately. If the operation could not be done, it says so (through a return value) or schedules the action for later. Nonblocking operations are **functions** or scheduling statements.

The complete picture

Mechanism	Blocking form	Nonblocking form
Mailbox: add	`put()` (blocks when a bounded mailbox is full)	`try_put()` returns 0 when full
Mailbox: remove	`get()` (blocks when empty)	`try_get()` returns 0 when empty
Mailbox: look	`peek()` (blocks when empty)	`try_peek()` returns 0 when empty
Semaphore: acquire	`get()` (blocks when keys are not available)	`try_get()` returns 0 when not available
Semaphore: release	None	`put()` never blocks
Event: wait	`@(ev)`, `wait (ev.triggered)`	Test `ev.triggered` in an `if`
Event: trigger	None (`->` and `->>` never suspend the caller)	`-> ev` acts now, `->> ev` acts in the NBA region
Threads	`fork ... join`, `fork ... join_any`, `wait fork`	`fork ... join_none`
Time	`#delay`, `@`, `wait`	None
Assignment (for comparison)	`a = b;` takes effect immediately	`a <= b;` takes effect in the NBA region

When each is appropriate

Use blocking when	Use nonblocking when
The thread has nothing useful to do until the operation completes (a driver waiting for the next transaction)	The thread must not stall (a monitor that must sample every clock edge)
You want automatic flow control	The code is in a function, which cannot block
You want the simplest code	You need to do something else if the resource is not available, or to poll several sources

Use blocking when	Use nonblocking when
	You are emptying a mailbox at the end of the test

The danger on each side

Style	Danger	Protection
Blocking	The thread may wait forever (deadlock)	Timeouts built with `fork` ... `join_any` and `disable fork`; a global watchdog
Nonblocking	The return value is ignored, or the call is repeated in a loop that consumes no time	Always test the return value; always include a time-consuming statement in a polling loop

```
// A monitor-style loop that must never stall: nonblocking put with a check
forever begin
  @(posedge clk);
  if (sample_valid) begin
    txn = new();
    capture(txn);
    if (!mon2scb_mbx.try_put(txn))
      $error("mon2scb_mbx full: scoreboard is not keeping up");
  end
end
```

Interview questions

Q1. What is the difference between blocking and nonblocking mailbox methods? The blocking methods `put`, `get` and `peek` are tasks that suspend the caller until the operation can complete. The nonblocking methods `try_put`, `try_get` and `try_peek` are functions that return immediately with a status: positive for success and 0 when the operation could not be performed.

Q2. Why can a function call `try_get()` but not `get()`? A function must execute in zero simulation time and cannot contain anything that might suspend it. `get()` is a task that may block; `try_get()` is a function that never does.

Q3. Is `-> ev` a blocking statement? No. Despite sometimes being called the "blocking trigger", it does not suspend the process. The name distinguishes it from `->>`, whose effect is deferred in the same way as a nonblocking assignment.

6.4 Race Conditions

Definition

A **race condition** exists when the result of a simulation depends on the order in which the simulator executes threads that are ready at the same simulation time. The standard leaves that order undefined, so:

- two simulators may give different results for the same code;
- the same simulator may give different results after a change of options, optimisation level or version, or after an unrelated edit that reorders the source;
- the testbench may behave differently in RTL and in gate-level simulation.

A race is a bug even if the code "works" today.

The kinds of race and their cures

Kind	Example	Cure
Read/write of a shared variable in the same time step	One thread assigns `data` at time 10; another reads `data` at time 10	Separate them in time or region: nonblocking assignment, a clocking block, a mailbox
Write/write	Two threads assign the same variable at the same time; the last one wins	One owner per variable; a semaphore
Event trigger versus wait	`-> ev` and `@(ev)` at the same time	`wait (ev.triggered)` or `->>`
Testbench versus DUT at a clock edge	The testbench changes a DUT input at `posedge clk` with a blocking assignment, while the DUT samples it at the same edge	Drive through a clocking block or with nonblocking assignments; sample before the edge (Module 8)
Lost update (read-modify-write)	Two threads read a counter, both add one, both write back	A semaphore around the whole read-modify-write
Order of `initial` blocks	One `initial` block uses an object that another constructs at time 0	One `initial` block controls the order; or wait for an explicit event

Example 1: read/write at the same time (simulator-dependent)

```
logic [7:0] data = 8'h00;

initial #10 data = 8'hAA;                              // thread 1 writes at time 10
initial #10 $display("[%0t] data = %h", $time, data);   // thread 2 reads at time 10
```

The printed value is `aa` if the simulator runs thread 1 first and `00` if it runs thread 2 first. Both are correct according to the standard, so no single "expected output" exists.

Ways to make the result defined:

Change	Result	Why
Write with `data <= 8'hAA;`	Always `00`	The update happens in the NBA region, after all Active-region reads
Read with `$strobe` instead of `$display`	Always `aa`	`$strobe` prints in the Postponed region, after all updates
Read at `#11`, or on the opposite clock edge	Always `aa`	The two actions are at different times

Example 2: the testbench-DUT race at a clock edge

```
// DUT
always_ff @(posedge clk) q <= d;

// Testbench: RACE
initial begin
  @(posedge clk);
  d = 1;              // blocking assignment at the same edge at which the DUT samples d
end
```

Both the `always_ff` and the `initial` block wake at the same clock edge. If the testbench runs first, the DUT samples the new `d` (1) at this edge. If the DUT runs first, it samples the old `d` and sees the 1 one cycle later. The DUT's behaviour changes by a whole clock cycle depending on the simulator.

Cures: drive with a nonblocking assignment (`d <= 1;`), so that `d` changes after the DUT has sampled it; or drive on the inactive clock edge; or, the proper method, drive and sample through a **clocking block**, which defines exactly when inputs are sampled and outputs are driven relative to the clock. Clocking blocks are the subject of Module 8.

Example 3: lost update (deterministic) and its cure

This race does not depend on the simulator at all. It comes from a thread being suspended in the middle of a read-modify-write sequence.

tb_race_lost_update.sv

```
module tb_race_lost_update;

  int       shared_count = 0;
  semaphore key          = new(1);

  task automatic add_unsafe();
    int tmp;
    tmp = shared_count;             // read
    #1;                             // suspended in the middle (models work that takes time)
    shared_count = tmp + 1;         // write back
  endtask

  task automatic add_safe();
    int tmp;
    key.get();                      // only one thread at a time from here ...
    tmp = shared_count;
    #1;
    shared_count = tmp + 1;
    key.put();                      // ... to here
  endtask

  initial begin
    fork
      add_unsafe();
      add_unsafe();
    join
    $display("[%0t] unsafe: shared_count = %0d (expected 2)", $time, shared_count);

    shared_count = 0;
    fork
      add_safe();
      add_safe();
    join
    $display("[%0t] safe  : shared_count = %0d (expected 2)", $time, shared_count);
  end

endmodule
```

EXPECTED OUTPUT

```
[1] unsafe: shared_count = 1 (expected 2)
[3] safe  : shared_count = 2 (expected 2)
```

Execution flow

Time	Unsafe version	
0	Thread 1 reads 0 into its `tmp`, suspends	Thread 2 reads 0 into its `tmp`, suspends

Time	Unsafe version	
1	Thread 1 writes 0 + 1 = 1	Thread 2 writes 0 + 1 = 1. One increment is lost

Time	Safe version	
1	Thread 1 takes the key, reads 0, suspends	Thread 2 blocks in `key.get()`
2	Thread 1 writes 1, returns the key	Thread 2 takes the key, reads 1, suspends
3		Thread 2 writes 2, returns the key

(Which of the two threads is "thread 1" is up to the simulator. The result is the same either way.)

The tasks are declared `automatic` so that each call has its own `tmp`. With a static task both threads would share one `tmp`, which is a second bug of the same family (Module 3).

Why #0 is not a cure

Adding `#0` before the read in Example 1 moves that thread to the Inactive region, so the write (in the Active region) happens first. It appears to work. But if the writer is later changed to use `#0` as well, or is woken by a nonblocking update, the race is back, and the code now contains unexplained `#0`s. `#0` documents nothing and guarantees little. Use a mechanism that states the intended order.

> **How to say it in an interview:** "A race condition is when the result depends on the execution order of processes scheduled at the same simulation time, which the LRM leaves undefined. Typical cases are a blocking write and a read of the same variable at the same clock edge, and triggering an event in the same time step in which another process starts to wait for it. I avoid them by using nonblocking assignments and clocking blocks between the testbench and the DUT, `wait (ev.triggered)` or `->>` for events, mailboxes for passing data, and semaphores for shared resources."

Interview questions

Q1. What is a race condition? Give an example. A situation in which the simulation result depends on the order in which processes that are active at the same simulation time are executed. Example: one process assigns `a = 1` at a clock edge with a blocking assignment while another process reads `a` at the same edge.

Q2. How do nonblocking assignments avoid races in RTL? All right-hand sides are evaluated in the Active region using the old values, and all left-hand sides are updated afterwards in the NBA region. Every flip-flop therefore sees the pre-edge values, whatever the order of the `always` blocks.

Q3. How do you avoid a race between the testbench and the DUT? Drive DUT inputs and sample DUT outputs through a clocking block (with input and output skews), or drive with nonblocking assignments and sample before the clock edge, or use the opposite clock edge. Historically, placing the testbench in a `program` block, which executes in the Reactive region, served the same purpose.

Q4. Two threads each execute `count = count + 1;` at the same simulation time, with no delay in between. Is an increment lost? No. A thread runs until it blocks, and this statement contains no blocking point, so each increment is completed before the other thread runs. The final value is correct, although the order is undefined. An update is lost only when a thread can be suspended between its read and its write.

Q5. Is `#0` a good way to fix a race? No. It reorders execution within the time step by moving a process to the Inactive region, but it does not express the dependency, it fails as soon as another process also uses `#0`, and it makes the code hard to reason about.

6.5 Deadlocks

Definition

A **deadlock** is a state in which two or more threads are each blocked waiting for something that can only be provided by another of the blocked threads. None of them can ever continue. More loosely, any thread that is blocked forever on a condition that will never occur is called deadlocked (or "hung").

The classic case: two semaphores taken in opposite orders

tb_deadlock.sv

```
module tb_deadlock;

  semaphore key_a = new(1);
  semaphore key_b = new(1);

  initial begin : thread_1
    key_a.get();
    $display("[%0t] T1 holds A", $time);
    #5;
    $display("[%0t] T1 waits for B", $time);
    key_b.get();                                  // blocked: T2 holds B
    $display("[%0t] T1 holds A and B", $time);    // never reached
    key_b.put();
    key_a.put();
  end

  initial begin : thread_2
    #1;
    key_b.get();
    $display("[%0t] T2 holds B", $time);
    #5;
    $display("[%0t] T2 waits for A", $time);
    key_a.get();                                  // blocked: T1 holds A
    $display("[%0t] T2 holds A and B", $time);    // never reached
    key_a.put();
    key_b.put();
  end

  initial begin : watchdog
    #100;
    $display("[%0t] WATCHDOG: no progress, the threads are deadlocked", $time);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[0] T1 holds A
[1] T2 holds B
[5] T1 waits for B
[6] T2 waits for A
[100] WATCHDOG: no progress, the threads are deadlocked
```

```
        holds                        waits for
T1  ---------->  key A   <----------------------  T2
 |                                                ^
 |   waits for                          holds     |
 +------------------>  key B  --------------------+

A cycle: T1 waits for what T2 holds, and T2 waits for what T1 holds.
```

The cure: every thread acquires the semaphores **in the same order**. If thread 2 also takes A first and then B, it simply blocks on A at time 1 until thread 1 has finished with both, and no cycle can form.

```
initial begin : thread_2_fixed
  #1;
  key_a.get();        // same order as thread 1: A first ...
  key_b.get();        // ... then B
  // ... use both resources ...
  key_b.put();
  key_a.put();
end
```

Other ways a testbench hangs

Situation	Why nothing proceeds	Cure
`@(ev)` after the event has already been triggered	The waiter missed the only trigger	`wait (ev.triggered)`, `->>`, or a persistent flag
Waiting on an event that was never connected between two classes	Each object has its own private event	Share the handle before forking
`get()` on a mailbox whose producer has finished or died	No message will ever arrive	End-of-test logic; `try_get`; a timeout
`put()` on a full bounded mailbox whose consumer has stopped	No space will ever be freed	Same
Two threads connected by two bounded mailboxes, each blocked in `put` towards the other	A cycle of full mailboxes	Unbounded mailbox in one direction; redesign the protocol
A semaphore key not returned on an error path, or its holder killed	No key will ever be put	Release on every path; do not kill threads in critical sections
`fork` ... `join` containing a `forever` thread	`join` waits for a thread that never ends	`join_any` / `join_none`
`wait fork` with a never-ending background child	Same	Guard thread
Generator waits for `drv_done`; driver waits for a second transaction before triggering it	Each waits for the other (a protocol deadlock)	Review the handshake
Waiting for a DUT signal that never comes because of a DUT bug	The testbench waits forever	A timeout that reports an error

How a deadlock appears in practice

What you see	What it means
The simulation ends early, quietly, with no error and with the test incomplete	Every thread is blocked and nothing is scheduled, so the simulator has no events left and stops. This happens when the clock is not free-running, or is generated by a thread that has ended

What you see	What it means
The simulation runs until the maximum time or the wall-clock limit of the regression	The clock generator keeps the simulation alive while all the useful threads are blocked
Simulation time stops advancing and the simulator uses 100% CPU	Not a deadlock but a zero-delay loop (for example `wait (ev.triggered)` or `try_get` in a loop without a delay)

The first symptom is the dangerous one: a test that stops early without printing an error can be reported as "passed" by a careless script. A test should print an explicit end-of-test message that the regression script requires, and should count the transactions it expected to check.

Prevention

Technique	How
Global watchdog	`initial begin #(MAX_TIME); $fatal(1, "watchdog timeout"); end` in the testbench top. Every testbench should have one
Timeout on each important wait	`fork begin fork wait_for_x(); #limit $error("timeout"); join_any disable fork; end join`
Fixed acquisition order	Always take semaphores in the same order
Release on all paths	One `put()` for every `get()`, including error paths
Nonblocking alternatives	`try_get()` where waiting forever is not acceptable
Simple ownership	One mailbox per direction, constructed once; one owner per resource
End-of-test condition	The test ends on a positive condition ("all N transactions checked"), not on "nothing seems to be happening"

> **Interview trap:** "The simulation finished at time 350 with no errors, but only two of the ten transactions were checked. What happened?" The testbench deadlocked, or a thread died, and with nothing left to schedule the simulator ended normally. "No errors" is not "passed". Look for the last message of each component to find which thread is blocked and on what.

Interview questions

Q1. What is a deadlock? A situation in which two or more threads are blocked, each waiting for a resource or an action that only another blocked thread can provide, so that none of them can ever resume.

Q2. Give an example of a deadlock with semaphores and its solution. Thread 1 takes semaphore A and then requests B; thread 2 takes B and then requests A. Each holds what the other needs. The solution is for all threads to acquire the semaphores in the same order.

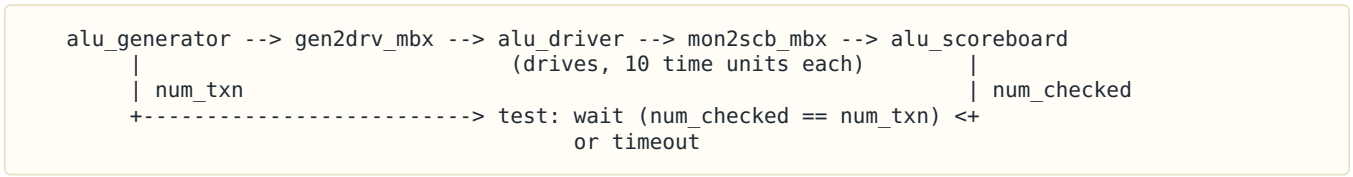
Q3. How do you debug a hanging simulation? First decide whether time is advancing. If time is frozen, a zero-delay loop exists: look for loops without a blocking statement. If time advances or the simulation ends early, a thread is blocked: use the last messages printed by each component, or the simulator's thread/process view, to find the blocking statement (`get`, `@`, `wait`), and then ask who was supposed to provide the message, trigger or key.

Q4. How do you protect a testbench against hangs? A global watchdog that ends the simulation with a fatal error after a maximum time, timeouts on individual waits built with `fork ... join_any` and `disable fork`, and an explicit end-of-test condition.

Q5. What is the difference between a deadlock and a race condition? A race is a result that depends on undefined execution order: the simulation completes but may be wrong. A deadlock is a state from which threads can never continue: the simulation hangs or ends prematurely. An event race (a missed trigger) frequently causes a hang, so the two often appear together.

6.6 Synchronization Between Multiple Processes: a Complete Example

The example connects three components with two mailboxes and ends the test on a positive condition, protected by a timeout. It is a miniature of the environment built in Module 9. To stay self-contained it has no DUT: the driver computes the result itself and forwards the transaction, standing in for the DUT and the monitor.



tb_multi_process_sync.sv

```
module tb_multi_process_sync;

  class alu_txn;
    bit [7:0]    a, b;
    logic [8:0]  result;
    int unsigned id;
  endclass

  class alu_generator;
    mailbox #(alu_txn) gen2drv_mbx;
    int unsigned       num_txn = 3;

    task run();
      alu_txn txn;
      for (int unsigned i = 1; i <= num_txn; i++) begin
        txn    = new();
        txn.id = i;
        txn.a  = 8'(i);
        txn.b  = 8'(2 * i);
        $display("[%0t] GEN: put id=%0d", $time, txn.id);
        gen2drv_mbx.put(txn);
      end
    endtask
  endclass

  class alu_driver;
    mailbox #(alu_txn) gen2drv_mbx;
    mailbox #(alu_txn) mon2scb_mbx;

    task run();
      alu_txn txn;
      forever begin
        gen2drv_mbx.get(txn);
        #10;                                    // one transfer takes 10 time units
        txn.result = txn.a + txn.b;             // stands in for the DUT
        $display("[%0t] DRV: drove id=%0d, result=%0d", $time, txn.id, txn.result);
        mon2scb_mbx.put(txn);                   // stands in for the monitor
      end
    endtask
```

```
  endclass

  class alu_scoreboard;
    mailbox #(alu_txn) mon2scb_mbx;
    int unsigned       num_checked;

    task run();
      alu_txn txn;
      forever begin
        mon2scb_mbx.get(txn);
        if (txn.result !== txn.a + txn.b)
          $display("[%0t] SCB: id=%0d FAIL", $time, txn.id);
        else
          $display("[%0t] SCB: id=%0d PASS", $time, txn.id);
        num_checked++;
      end
    endtask
  endclass

  mailbox #(alu_txn) gen2drv_mbx, mon2scb_mbx;
  alu_generator      gen;
  alu_driver         drv;
  alu_scoreboard     scb;

  initial begin
    // build
    gen2drv_mbx = new();
    mon2scb_mbx = new();
    gen = new();
    drv = new();
    scb = new();

    // connect
    gen.gen2drv_mbx = gen2drv_mbx;
    drv.gen2drv_mbx = gen2drv_mbx;
    drv.mon2scb_mbx = mon2scb_mbx;
    scb.mon2scb_mbx = mon2scb_mbx;

    // run
    fork
      gen.run();
      drv.run();
      scb.run();
    join_none

    // end of test: all transactions checked, or timeout
    fork
      begin
        fork
          wait (scb.num_checked == gen.num_txn);
          begin
            #1000;
            $display("[%0t] TEST: TIMEOUT", $time);
          end
        join_any
        disable fork;
      end
    join

    $display("[%0t] TEST: %0d of %0d transactions checked",
             $time, scb.num_checked, gen.num_txn);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[0] GEN: put id=1
[0] GEN: put id=2
[0] GEN: put id=3
[10] DRV: drove id=1, result=3
[10] SCB: id=1 PASS
[20] DRV: drove id=2, result=6
[20] SCB: id=2 PASS
[30] DRV: drove id=3, result=9
[30] SCB: id=3 PASS
[30] TEST: 3 of 3 transactions checked
```

What synchronizes what

Mechanism in the example	Threads involved	Purpose
`gen2drv_mbx` (unbounded)	Generator → driver	The generator runs ahead and finishes at time 0; the mailbox holds the three transactions until the driver takes them
`mon2scb_mbx`	Driver → scoreboard	Each completed transaction is checked as soon as it arrives
`fork` ... `join_none`	Top → three components	Starts the components; the driver and scoreboard loop forever
`wait (scb.num_checked == gen.num_txn)`	Scoreboard → test	A positive end-of-test condition. The test does not end when the generator finishes (time 0); it ends when the last transaction has been checked (time 30)
`fork` ... `join_any` with `disable fork` in a guard	Test	Timeout. Without the guard, `disable fork` would also kill the three components

Points to notice

- The driver prints **before** it puts the transaction into `mon2scb_mbx`, and the scoreboard prints **before** it increments `num_checked`. Each line of output is therefore caused by the previous one, and the order does not depend on the simulator.
- The generator finishes at time 0. A test that ended on "generator done" would stop before anything had been driven. This is the most common end-of-test bug.
- The driver passes the *same* object on to the scoreboard. In a real environment the monitor creates a **new** transaction from what it observes on the interface, so that the checked data comes from the DUT pins and not from the stimulus.

6.7 Event versus Mailbox versus Semaphore

Comparison table

Property	Event	Mailbox	Semaphore
Purpose	Notify that something happened	Transfer data	Control access to a shared resource
What it holds	Nothing	A FIFO of messages	A count of keys

Property	Event	Mailbox	Semaphore
Carries data	No	Yes	No
Kind of object	Built-in synchronization object, handle semantics	Built-in class	Built-in class
Needs `new()`	No	Yes	Yes
Producing operation	`-> ev`, `->> ev`	`put()`, `try_put()`	`put()`
Consuming operation	`@(ev)`, `wait (ev.triggered)`	`get()`, `peek()`, `try_get()`, `try_peek()`	`get()`, `try_get()`
Is the state remembered	No (only to the end of the time step, through `triggered`)	Yes, until a `get` removes the message	Yes, until a `get` takes the key
Counts occurrences	No	Yes, one message each	Yes, one key each
Number of threads woken by one action	**All** waiters	One (the message goes to a single getter)	As many as the returned keys satisfy
Can the producer block	No	Yes, if bounded and full	No
Can the consumer block	Yes	Yes, if empty	Yes, if keys are not available
Nonblocking consumer call	Test `ev.triggered`	`try_get()`, `try_peek()`	`try_get()`
Query	`ev.triggered`	`num()`	None
Typical use	`reset_done`, `drv_done`, `gen_done`	generator → driver, monitor → scoreboard	Two sequences sharing one driver or interface

Choosing

```
Do the threads need to pass data?
     yes ------------------------------------------------> MAILBOX
     no
      |
Do they compete for something only one (or N) may use at a time?
     yes ------------------------------------------------> SEMAPHORE
     no
      |
Does one thread just need to tell others "it has happened"?
     yes, and all waiters are already waiting ---------> EVENT
     yes, but a waiter may arrive later ---------------> flag variable,
                                                         semaphore with 0 keys,
                                                         or mailbox
```

They are used together

A real testbench uses all three at once. In the ALU environment: mailboxes carry `alu_txn` handles between generator, driver, monitor and scoreboard; the `drv_done` and `gen_done` events pace the generator and end the test; and a semaphore protects the interface whenever more than one source of stimulus (for example a main sequence and an error-injection sequence) shares the driver.

Interview questions

Q1. Compare event, mailbox and semaphore. An event signals that something has occurred; it carries no data, is not remembered, and wakes all waiting threads. A mailbox passes data between threads as a FIFO of

messages, with blocking `get` and optionally blocking `put`. A semaphore holds keys and controls access to a shared resource; a thread blocks in `get` until a key is available.

Q2. Which would you use for each of these: sending transactions from monitor to scoreboard; telling the test that the generator has finished; letting two sequences use one driver? A mailbox; an event; a semaphore.

Q3. Which of the three can lose information, and how? An event. A trigger that occurs when nobody is waiting is lost, and two triggers before the waiter runs are seen as one. A mailbox stores every message and a semaphore counts every key.

Q4. Which of the three need `new()`? Mailbox and semaphore, because they are classes. An event does not.

Q5. Can a mailbox be used in place of an event or a semaphore? Yes. A message with no meaningful content acts as a remembered notification, and a bounded mailbox of size 1 acts as a lock. It is rarely done, because the dedicated constructs state the intention clearly, but it shows that the mailbox is the most general of the three.

Part 7: Module 7 Revision and Practice

7.1 Revision Summary

The whole module on one page

Construct	Purpose	Create	Produce	Consume	Blocks when
`event`	Notification	`event ev;`	`-> ev;` `->> ev;`	`@(ev);` `wait (ev.triggered);`	Waiting for a trigger
`mailbox`	Data transfer	`mbx = new();` or `new(N)`	`put` / `try_put`	`get` / `peek` / `try_get` / `try_peek`	`get`, `peek`: empty. `put`: bounded and full
`semaphore`	Resource sharing	`sem = new(N);`	`put(n)`	`get(n)` / `try_get(n)`	`get`: not enough keys
`fork` ... `join`	Parallel threads, wait for all				Until all children finish
`fork` ... `join_any`	Wait for the first				Until one child finishes
`fork` ... `join_none`	Do not wait				Never

Rules to remember

Processes

- The simulator runs one thread at a time; a thread runs until it blocks.
- The order of threads that are ready at the same time is undefined. Code that depends on it has a race.
- A loop with no blocking statement hangs the simulation at the current time.

Events

- `-> ev` wakes only the threads already waiting. It is not remembered.
- `@(ev)` is edge-sensitive; `wait (ev.triggered)` is level-sensitive until the end of the time step.
- `->> ev` fires in the NBA region; neither `->` nor `->>` suspends the caller.
- The unsafe pair is `->` with `@` in the same time step.
- `wait (ev.triggered)` in a zero-time loop spins forever.
- Events are handles: `b = a` merges them; a thread already waiting on the old `b` never wakes; triggering `null` does nothing.
- `wait_order (a, b, c) else ...;` checks the order of events.

Fork and join

- Each statement in a `fork` is one thread; `begin` ... `end` groups statements into one thread.
- `join`: all. `join_any`: first. `join_none`: no wait, and the children start only when the parent blocks.
- Unfinished threads are never killed automatically.

- `disable fork` kills **all descendants of the calling process**. Guard it: `fork begin ... disable fork; end join`.
- `wait fork` waits for all immediate children of the calling process.
- Loop plus `join_none`: copy the loop variable into an `automatic` variable declared outside the child's `begin` ... `end`.
- Only `join_none` is legal in a function.

Mailbox

- A class: needs `new()`. `new()` or `new(0)` is unbounded; `new(N)` is bounded.
- `put`, `get`, `peek` are blocking tasks. `try_put`, `try_get`, `try_peek` are nonblocking functions returning positive, 0, or (typeless mailbox only) negative.
- `get` removes; `peek` copies.
- Use `mailbox #(T)`: type errors at compile time instead of run time.
- A mailbox of objects stores handles. Create a new object for every `put`.
- One mailbox, constructed once, shared by handle.

Semaphore

- A class: needs `new(keys)`; the default is 0 keys.
- `get(n)` blocks (task); `put(n)` never blocks (function); `try_get(n)` returns positive or 0.
- No ownership, no key query, no upper limit: a stray `put` adds a key.
- One `put` for every `get`, on every path.

Timing, races, deadlocks

- `#` waits for time, `@` for a change, `wait` for a level.
- Races: cured by nonblocking assignments, clocking blocks, `wait (ev.triggered)`, `->>`, mailboxes, semaphores. Not by `#0`.
- Deadlocks: cured by a fixed acquisition order, releasing on all paths, timeouts and a global watchdog.
- End a test on a positive condition, not when the generator finishes.

The key comparisons

Comparison	One-line answer
`join` / `join_any` / `join_none`	Parent waits for all / the first / none
`@(ev)` / `wait (ev.triggered)`	Edge / level within the time step
`->` / `->>`	Trigger now / trigger in the NBA region
`get` / `peek`	Remove / copy
`get` / `try_get`	Blocking task / nonblocking function
Bounded / unbounded	`put` can block / `put` never blocks
Typeless / parameterized mailbox	Run-time / compile-time type check
`disable fork` / `disable label`	Descendants of the caller / a named block wherever it runs
`wait fork` / `join`	All children of the process / the threads of one block
Mailbox / semaphore	Data / keys

Comparison	One-line answer
Event / mailbox / semaphore	Notify / transfer / protect
Race / deadlock	Wrong order / no progress

7.2 Interview Questions and Answers

The questions inside Parts 1 to 6 cover each topic directly. These are additional.

Basic level

Q1. Name the IPC mechanisms of SystemVerilog. Events, mailboxes and semaphores, together with `fork` ... `join` / `join_any` / `join_none`, `wait fork` and `disable fork` for creating and controlling threads.

Q2. What is the difference between a thread created by `initial` and one created by `fork`? An `initial` block creates one static process per block at time 0, at module level. A `fork` creates dynamic processes at run time, anywhere procedural code can appear, including inside class methods.

Q3. How many threads does `fork begin a(); b(); end c(); join` create? Two.

Q4. Which form of fork does not block the parent? `join_none`.

Q5. How do you declare and trigger an event? `event done;` and `-> done;`.

Q6. What does `mbx.num()` return? The number of messages currently in the mailbox.

Q7. How do you create a mailbox that can hold at most five transactions? `mailbox #(alu_txn) mbx = new(5);`

Q8. What does `sem = new(2);` create? A semaphore with two keys.

Q9. What happens when `get()` is called on an empty mailbox? The calling thread blocks until another thread puts a message.

Q10. What is the return value of `try_get()` on an empty mailbox? 0.

Q11. Does an event carry data? No.

Q12. Which IPC constructs must be constructed with `new()`? Mailbox and semaphore.

Q13. What does `wait fork` do? It blocks until all child threads of the calling process have finished.

Q14. What is mutual exclusion? A guarantee that only one thread at a time uses a shared resource.

Intermediate level

Q15. When do the threads of a `fork` ... `join_none` start executing? When the parent thread reaches a blocking statement or terminates, not at the `join_none` itself.

Q16. Why is `automatic` needed for a variable used by threads forked in a loop? So that each iteration, and therefore each thread, gets its own variable. A static variable, or the loop variable itself, exists only once, and all threads would see its final value.

Q17. What is the difference between `wait (ev.triggered)` and testing `if (ev.triggered)`? `wait` blocks until the event has been triggered in the current time step. `if` only tests whether it has been triggered so far in this time step and never blocks.

Q18. A generator and a driver are connected by an unbounded mailbox, with no handshake. The generator creates 1000 transactions. When does the generator finish? At the simulation time it started, because `put` on an unbounded mailbox never blocks. All 1000 transactions sit in the mailbox while the driver works through them.

Q19. What is wrong with ending the test when the generator has finished? The transactions may still be in the mailbox or in the DUT. The driver, monitor and scoreboard need time to process them. The test should wait until the scoreboard has checked the expected number, or for a drain time.

Q20. How do you pass a mailbox to a class? As a handle: declare a mailbox property in the class and assign it from outside, or pass it as a constructor argument. The mailbox object itself is constructed once, by the owner of both ends.

Q21. What is the difference between `peek()` followed by `get()`, and `get()` alone? The result for the consumer is the same message. The difference is in what the producer sees: between `peek` and `get` the message is still in the mailbox, so a bounded mailbox stays full and the producer stays blocked. This is used to hold the generator back until the driver has finished an item.

Q22. Can two threads wait on the same event? On the same mailbox? On the same semaphore? Yes to all. One event trigger wakes all waiters. One mailbox message goes to one waiter. Returned semaphore keys go to as many waiters as they satisfy.

Q23. What is the effect of `disable fork` in a thread that has no children? Nothing.

Q24. How do you implement "wait for either event A or event B"? `@(a or b);`, or `fork @(a); @(b); join_any disable fork;` inside a guard when the two branches do more than wait.

Q25. Why is a parameterized mailbox preferred to `mailbox`? The element type is checked at compile time and documents what the connection carries. A typeless mailbox fails at run time when a message is retrieved into a variable of the wrong type.

Q26. Is `sem.put()` allowed before any `get()`? Yes. It increases the key count. For a mutex this is a bug, since it creates a second key.

Advanced and scenario-based

Q27. A test passes on one simulator and hangs on another. Where do you look first? For races, especially an event that is triggered with `->` and waited for with `@` in the same time step. One simulator happens to run the waiter first; the other runs the trigger first and the waiter then waits forever. Also check for threads forked in a loop without an automatic copy, and for `initial` blocks that depend on each other's order at time 0.

Q28. After a timeout was added to one test, the scoreboard stopped reporting anything from that point on. Why? The timeout uses `fork` ... `join_any` followed by `disable fork` without a guard. `disable fork` killed every descendant of the calling thread, including the monitor and scoreboard threads that had been started earlier from the same thread. Wrap the timeout in `fork begin ... end join`.

Q29. A driver receives ten transactions that are all identical to the last one generated. Why? The generator constructs one transaction object outside its loop and puts the same handle ten times. All ten mailbox entries refer to one object, which holds the values of the last randomization. A new object must be created, or a copy put, for each transaction.

Q30. A simulation ends at a small simulation time with no error message, but the test did not complete. What happened? All threads were blocked and no further events were scheduled, so the simulator stopped: a deadlock or a dead thread. Typical causes are a missed event trigger, a `get` on a mailbox that nobody writes (for example two separately constructed mailboxes), or a semaphore key never returned.

Q31. Two sequences drive the same interface and occasionally the bus shows a mixture of the two transfers. The code uses a semaphore. What could still be wrong? The two users may each have constructed their own semaphore; one path may use the bus without taking the key; a stray `put()` may have added a second key; or the critical section may be too short (the key is returned before the transfer has really finished on the pins).

Q32. How would you make a generator wait for the driver without using an event? Create the mailbox with a bound of 1 and have the driver `peek`, drive, and only then `get`. The generator's next `put` blocks until the `get`.

Q33. What does this do: `fork forever @(posedge clk) count++; join_none` followed later by `wait fork;`? `wait fork` never returns, because the `forever` thread is a child of the same process and never ends.

Q34. How can a thread stop one particular other thread without affecting anything else? Through a `process` handle: the target thread stores `process::self()` in a variable, and the controlling thread calls `kill()` on it. This is covered in Module 11.

Q35. Why are mailboxes preferred over events plus shared variables for passing transactions? An event plus a variable holds one item and loses any item produced while the consumer is busy; it also exposes the event race. A mailbox queues every item in order and has blocking built in on both sides.

Q36. In which scheduling region does a thread woken by `->>` run? The event fires in the NBA region; the woken thread is then executed in the next Active region of the same time slot.

Q37. How does UVM relate to these constructs? UVM is written in SystemVerilog and builds on them. TLM ports and FIFOs (`uvm_tlm_fifo`, analysis ports) replace hand-connected mailboxes; the sequencer-driver handshake replaces the mailbox plus `drv_done` event; `uvm_event` and `uvm_barrier` extend events; objections replace the hand-written end-of-test condition; sequencer `lock`/`grab` replaces a semaphore. Phases are started with `fork` internally.

7.3 Output-Prediction Questions

Work out the output on paper before reading the answer. Each example is a complete module.

P1.

```
module p1;
  initial begin
    fork
      #5  $display("A");
      #15 $display("B");
    join_any
    $display("C at %0t", $time);
    #20 $display("D at %0t", $time);
  end
endmodule
```

Answer: `A`, `C at 5`, `B`, `D at 25`. `join_any` releases the parent at 5. Thread B is not killed and prints at 15. The parent prints again at 5 + 20 = 25.

P2.

```
module p2;
  initial begin
    $display("1");
    fork
      $display("2");
    join_none
    $display("3");
    #0 $display("4");
  end
endmodule
```

Answer: `1`, `3`, `2`, `4`. The child cannot start until the parent blocks, so `3` comes before `2`. At `#0` the parent moves to the Inactive region; the child, in the Active region, runs first.

P3.

```
module p3;
  initial begin
    for (int i = 0; i < 3; i++)
      fork
        #(i) $display("i=%0d t=%0t", i, $time);
      join_none
  end
endmodule
```

Answer: `i=3 t=3` three times. The threads start after the loop has ended, when `i` is 3. Each waits 3 time units and prints 3.

P4.

```
module p4;
  event e;
  initial begin
    -> e;
    @(e);
    $display("got it");
  end
  initial #10 $display("end");
endmodule
```

Answer: only `end`. The trigger is over before `@(e)` starts waiting, so the first block waits forever. With `wait (e.triggered);` in place of `@(e);` the output would be `got it` and then `end`.

P5.

```
module p5;
  mailbox #(int) m = new(2);
  int v;
  initial begin
    $display("%s", m.try_put(1) ? "ok" : "full");
    $display("%s", m.try_put(2) ? "ok" : "full");
    $display("%s", m.try_put(3) ? "ok" : "full");
    m.get(v);
    m.peek(v);
    $display("%0d %0d", v, m.num());
  end
endmodule
```

Answer: `ok`, `ok`, `full`, `2 1`. The mailbox holds two messages. `get` removes 1. `peek` copies 2 without removing it, so one message remains.

P6.

```
module p6;
  semaphore s = new(1);
  initial begin
    fork
      begin
        s.get();
        $display("[%0t] T1 in", $time);
        #10;
        $display("[%0t] T1 out", $time);
        s.put();
      end
      begin
        #1 s.get();
        $display("[%0t] T2 in", $time);
        #10 s.put();
      end
      begin
        #2;
        if (s.try_get()) $display("[%0t] T3 in", $time);
        else             $display("[%0t] T3 gave up", $time);
      end
    join
    $display("[%0t] done", $time);
  end
endmodule
```

Answer: `[0] T1 in`, `[2] T3 gave up`, `[10] T1 out`, `[10] T2 in`, `[20] done`. T2 blocks at time 1 and is granted the key when T1 returns it at 10. T3 does not block: `try_get` fails at time 2 and it moves on.

P7.

```
module p7;
  initial begin
    fork
      begin #10 $display("A"); end
      begin #20 $display("B"); end
      begin #30 $display("C"); end
    join_any
    $display("first done at %0t", $time);
    #15 disable fork;
    #20 $display("end at %0t", $time);
  end
endmodule
```

Answer: `A`, `first done at 10`, `B`, `end at 45`. `disable fork` is executed at time 25. By then B has finished (at 20); only C is still running and is killed, so `C` is never printed.

P8.

```
module p8;
  event e;
  initial begin
    ->> e;
    $display("1");
    @(e);
    $display("2 at %0t", $time);
  end
endmodule
```

Answer: `1`, `2 at 0`. The nonblocking trigger fires in the NBA region of time 0, after the thread has started waiting. With `-> e` the thread would hang after printing `1`.

P9.

```
module p9;
  bit ready;
  initial begin
    ready = 1;
    #10 ready = 0;
    #10 ready = 1;
  end
  initial begin
    #5 wait (ready);
    $display("W %0t", $time);
  end
  initial begin
    #5 @(posedge ready);
    $display("E %0t", $time);
  end
endmodule
```

Answer: `W 5`, `E 20`. At time 5 `ready` is already 1: the level-sensitive `wait` passes at once, while the edge-sensitive `@` waits for the next rising edge, at time 20.

P10.

```
module p10;
  mailbox #(int) m = new(1);
  initial begin
    for (int i = 1; i <= 3; i++) begin
      m.put(i);
      $display("put %0d at %0t", i, $time);
    end
  end
  initial begin
    int v;
    repeat (3) begin
      #10 m.get(v);
    end
  end
endmodule
```

Answer: `put 1 at 0`, `put 2 at 10`, `put 3 at 20`. The mailbox holds one message, so each further `put` waits for the consumer's `get`, which occurs every 10 time units.

7.4 Find-the-Error and Debugging Questions

Compile-time and run-time errors

E1.

```
mailbox #(alu_txn) gen2drv_mbx;
initial begin
  alu_txn txn = new();
  gen2drv_mbx.put(txn);
end
```

The mailbox was never constructed: a null-handle run-time error at `put`. Add `gen2drv_mbx = new();`.

E2.

```
function void collect(mailbox #(alu_txn) mbx);
  alu_txn txn;
  mbx.get(txn);
endfunction
```

`get` is a blocking task and cannot be called from a function. Use `try_get` and test its return value, or make `collect` a task.

E3.

```
function void start_all();
  fork
    drv.run();
    mon.run();
  join_any
endfunction
```

Only `fork` ... `join_none` is allowed inside a function. Use `join_none` or change the function into a task.

E4.

```
event done;
initial @(posedge done) $display("done");
```

An event has no value and therefore no edges. Write `@(done)`.

E5.

```
mailbox mbx = new();
string  s = "abc";
int     i;
initial begin
  mbx.put(s);
  mbx.get(i);
end
```

A string was put and an `int` is requested: a run-time type error from the typeless mailbox. Declare `mailbox #(string)` so that the mistake is found at compile time.

E6.

```
semaphore sem = new();
initial begin
  sem.get();
  $display("in critical section");
  sem.put();
end
```

Not a compile error, but the thread hangs: `new()` creates zero keys. Write `new(1)`.

Run-time bugs (the code compiles and runs)

D1.

```
forever begin
  if (mbx.try_get(txn)) drive(txn);
end
```

When the mailbox is empty the loop repeats without blocking, and simulation time cannot advance. Use the blocking `mbx.get(txn);`, or add `@(posedge clk);` to the loop.

D2.

```
forever begin
  wait (start.triggered);
  num_starts++;
end
```

After the first trigger, `start.triggered` stays true for the rest of the time step, so the loop spins forever. Use `@(start);`.

D3.

```
foreach (drv[i])
  fork
    drv[i].run();
  join_none
```

All threads start after the loop, when `i` is past the last index: an out-of-range access (a null handle) instead of one thread per driver. Fix:
`foreach (drv[i]) begin automatic int k = i; fork drv[k].run(); join_none end`.

D4.

```
task send(alu_txn txn);
  bus_key.get();
  if (txn == null) begin
    $error("null transaction");
    return;
  end
  drive(txn);
  bus_key.put();
endtask
```

The error path returns while holding the key. Every later `get` blocks forever. Put the key back before `return`, or do the check before `get`.

D5.

```
class alu_generator;
  mailbox #(alu_txn) gen2drv_mbx = new();
endclass
class alu_driver;
  mailbox #(alu_txn) gen2drv_mbx = new();
endclass
```

Each class constructs its own mailbox. The generator fills one, the driver waits on the other. Construct one mailbox in the environment and pass its handle to both.

D6.

```
task run();
  fork
    mon.run();
  join_none
  fork
    wait_for_done();
    #5000 $error("timeout");
  join_any
  disable fork;
endtask
```

`disable fork` also kills `mon.run()`. Guard the timeout: `fork begin fork ... join_any disable fork; end join`.

D7.

```
// thread 1                          // thread 2
@(posedge clk);                      @(posedge clk);
-> sample_now;                       @(sample_now);
                                     sample();
```

Both threads wake at the same clock edge. If thread 1 runs first, thread 2 misses the trigger and samples one cycle late (at the next trigger). Use `wait (sample_now.triggered)` in thread 2 or `->> sample_now` in thread 1.

D8.

```
task run();
  alu_txn txn = new();
  forever begin
    @(vif.mon_cb);
    if (vif.mon_cb.valid_out) begin
      txn.result = vif.mon_cb.result;
      mon2scb_mbx.put(txn);
    end
  end
endtask
```

The monitor reuses one object. Every message in `mon2scb_mbx` refers to it, so the scoreboard sees later samples overwrite earlier ones. Move `txn = new();` inside the `if`.

D9.

```
fork
  gen.run();
  drv.run();      // forever loop
  mon.run();      // forever loop
join
$display("test finished");
```

`join` waits for threads that never end, so the message is never printed. Use `join_any` (the generator ends) followed by an end-of-test condition.

D10.

```
initial begin
  fork
    @(rst_done);
  join_none
  rst_done = other_done;
  -> rst_done;
end
```

The child may already be waiting on the original event object when the handle is reassigned (and in any case the two orders give different behaviour). A thread attached to the old object never wakes. Connect event handles before forking the threads that wait on them.

7.5 Coding Exercises with Solutions

Try each exercise before reading the solution.

Exercise 1: producer and consumer with a bounded mailbox

A producer sends the squares of 1 to 4 as fast as the mailbox allows. The mailbox holds one message. A consumer takes one value every 5 time units and prints it.

ex01_producer_consumer.sv

```
module ex01_producer_consumer;

  mailbox #(int) mbx = new(1);

  initial begin : producer
    for (int i = 1; i <= 4; i++)
      mbx.put(i * i);                       // blocks while the mailbox is full
  end

  initial begin : consumer
    int v;
    repeat (4) begin
      #5;
      mbx.get(v);
      $display("[%0t] consumer got %0d", $time, v);
    end
  end

endmodule
```

EXPECTED OUTPUT

```
[5] consumer got 1
[10] consumer got 4
[15] consumer got 9
[20] consumer got 16
```

The values arrive in the order they were sent. The producer needs no delay of its own: the bounded mailbox paces it.

Exercise 2: a reusable wait-with-timeout task

Write a task that waits for an event for at most a given time and reports whether the event occurred. It must not disturb any other thread of the caller.

ex02_timeout.sv

```
module ex02_timeout;

  event resp;

  task automatic wait_resp(input time limit, output bit ok);
    ok = 0;
    fork                                    // guard thread
      begin
        fork
          begin
            @(resp);
            ok = 1;
          end
          #limit;
        join_any
        disable fork;                       // stops only the two threads above
      end
    join
  endtask

  initial begin
    bit ok;

    fork
      #30 -> resp;                          // the response arrives at time 30
    join_none

    wait_resp(50, ok);
    $display("[%0t] first wait : ok = %b", $time, ok);

    wait_resp(50, ok);
    $display("[%0t] second wait: ok = %b", $time, ok);
  end

endmodule
```

EXPECTED OUTPUT

```
[30] first wait : ok = 1
[80] second wait: ok = 0
```

The first call ends at time 30 when the event arrives. The second call starts at time 30, after that trigger, sees no further trigger, and gives up after 50 time units, at time 80.

Exercise 3: ping-pong with two events

Two threads print `ping` and `pong` alternately, three times each, five time units apart. Stop both when the ping thread has finished.

ex03_ping_pong.sv

```
module ex03_ping_pong;

  event ping_e, pong_e;

  initial begin
    fork
      begin : ping_thread
        repeat (3) begin
          #5;
          $display("[%0t] ping", $time);
          -> ping_e;
          @(pong_e);
        end
      end

      begin : pong_thread
        forever begin
          @(ping_e);
          #5;
          $display("[%0t] pong", $time);
          -> pong_e;
        end
      end
    join_any
    disable fork;
    $display("[%0t] game over", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[5] ping
[10] pong
[15] ping
[20] pong
[25] ping
[30] pong
[30] game over
```

Each `@` is safe from the trigger race because the other thread always consumes time (`#5`) before triggering: the waiter is already waiting. `join_any` returns when the ping thread ends; `disable fork` stops the endless pong thread. No guard is needed here because this `initial` block has no other children.

Exercise 4: empty a mailbox from a function

Write a function that removes and prints everything in a mailbox and returns the number of messages removed.

ex04_drain.sv

```
module ex04_drain;

  mailbox #(int) mbx = new();

  function automatic int drain();
    int v;
    int n = 0;
    while (mbx.try_get(v)) begin            // a function may only use the try_ methods
      $display("  drained %0d", v);
      n++;
    end
    return n;
  endfunction

  initial begin
    int n;
    for (int i = 1; i <= 3; i++) mbx.put(i * 100);
    n = drain();
    $display("drained %0d messages, %0d left", n, mbx.num());
  end

endmodule
```

EXPECTED OUTPUT

```
  drained 100
  drained 200
  drained 300
drained 3 messages, 0 left
```

Exercise 5: limit the number of simultaneous jobs

Four jobs each need 10 time units. At most two may run at the same time. Job `k` asks to start at time `k + 1`.

ex05_job_pool.sv

```
module ex05_job_pool;

  semaphore pool = new(2);                  // two keys: two jobs at a time

  initial begin
    for (int i = 0; i < 4; i++) begin
      automatic int id = i;
      fork
        begin
          #(id + 1);
          pool.get();
          $display("[%0t] job %0d starts", $time, id);
          #10;
          $display("[%0t] job %0d ends", $time, id);
          pool.put();
        end
      join_none
    end
    wait fork;
    $display("[%0t] all jobs done", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[1] job 0 starts
[2] job 1 starts
[11] job 0 ends
[11] job 2 starts
[12] job 1 ends
[12] job 3 starts
[21] job 2 ends
[22] job 3 ends
[22] all jobs done
```

Jobs 2 and 3 ask for a key at times 3 and 4, find none, and wait in that order. Job 2 receives the key job 0 returns at 11; job 3 receives the key job 1 returns at 12.

Exercise 6: lock-step generator and driver without an event

Using only a mailbox, make the generator's `put` of item N+1 complete exactly when the driver has finished item N. The driver needs 10 time units per item.

ex06_lock_step.sv

```
module ex06_lock_step;

  mailbox #(int) gen2drv_mbx = new(1);      // bound of 1

  initial begin : generator
    for (int i = 1; i <= 3; i++) begin
      gen2drv_mbx.put(i);
      $display("[%0t] GEN: put %0d accepted", $time, i);
    end
  end

  initial begin : driver
    int item;
    forever begin
      gen2drv_mbx.peek(item);               // look: the mailbox stays full
      #10;                                  // drive the item
      gen2drv_mbx.get(item);                // remove: the generator may continue
    end
  end

endmodule
```

EXPECTED OUTPUT

```
[0] GEN: put 1 accepted
[10] GEN: put 2 accepted
[20] GEN: put 3 accepted
```

If the driver used `get` first and then the delay, the mailbox would be empty while item 1 was being driven, and the output would be `put 1` at 0, `put 2` at 0 and `put 3` at 10: the generator would be one item ahead.

Further practice (no solutions given)

1. Rewrite `tb_no_ipc` (Section 1.2) with a mailbox so that all four values are consumed. At what time is the last one read?
2. Write a task `wait_any(event a, event b, output int which)` that returns 1 or 2 according to the event that occurred first.
3. Build a module in which three threads each need the bus twice, protected by one semaphore, with request times chosen so that the output order is fully determined. Predict the output, then run it.

4. Implement your own mailbox class for `int` using a queue and `wait`, with `put`, `get`, `try_get` and `num`.
5. Create a deadlock with two bounded mailboxes and two threads, then remove it by changing one mailbox to unbounded.
6. Modify `tb_gen_drv_mailbox` so that the generator uses `wait (drv_done.triggered)` instead of `@(drv_done)`. Does it still work? What happens if the driver's `#10` is removed, and why?
7. Add a second driver to `tb_multi_process_sync` that takes transactions from the same `gen2drv_mbx`. Which driver gets which transaction, and what determines it?

7.6 Concepts You Must Understand Before Moving Forward

Processes

- [ ] I can explain how the simulator runs "parallel" threads one at a time, and what makes a thread block.
- [ ] I can explain why the order of threads at one simulation time is undefined and what a race is.

Events

- [ ] I can explain `->`, `->>`, `@(ev)` and `wait (ev.triggered)`, and draw the same-time-step race on a timeline.
- [ ] I know why `wait (ev.triggered)` must not be used in a zero-time loop.
- [ ] I can share an event between two classes and explain why the handles are connected before the threads are forked.
- [ ] I can write a `wait_order` with an `else` clause.

Fork and join

- [ ] I can state when the parent resumes for `join`, `join_any` and `join_none`, and when the children start.
- [ ] I can explain and fix the loop-variable bug.
- [ ] I can write a timeout with `join_any`, `disable fork` and the guard, and explain what the guard prevents.
- [ ] I know what `wait fork` waits for.

Mailbox

- [ ] I can list all eight methods and say which block and which are functions.
- [ ] I can explain bounded versus unbounded and typeless versus parameterized.
- [ ] I can explain why a new transaction object is needed for every `put`.
- [ ] I can write a generator and a driver connected by a mailbox, with a handshake.

Semaphore

- [ ] I can protect a shared resource with `get` and `put`.
- [ ] I know that `put` can add keys that were never taken, and why that matters.
- [ ] I can compare a semaphore with a mailbox and with an event.

Timing, races, deadlocks

- [ ] I can explain `#` versus `@` versus `wait` with an example.
- [ ] I can give three kinds of race and the cure for each.
- [ ] I can describe a deadlock, how it appears in a simulation, and how a watchdog protects against it.

- [ ] I can explain why a test should not end when the generator finishes.

7.7 What Comes Next

Module 8 covers interfaces, modports, virtual interfaces and clocking blocks. So far the driver has only pretended to drive (`#10`). With a virtual interface the driver class reaches the real DUT pins, and with a clocking block it does so without the testbench-versus-DUT race described in Section 6.4. Module 9 then assembles everything from Modules 4 to 8 into the complete layered testbench for the ALU, using exactly the mailbox, event and fork structures of this module.